

LogForge

Source Code Analysis and Mathematical Proof - Academic Paper

HLG to Apple Log: from color mathematics to reliable media delivery

Research object	Fixed revision and evidence
Repository	HongHuo0728 / LogForge
Version	1.2.1 (26927B)
Commit	6b43a93fbd4d1457ff20b4c99a030c0e398f520a
Study date	29 September 2026
Edition	Complete English translation of the Chinese research paper
Structure	24 chapters; 66 equations; 18 vector figures; 5 tables
Validation	Release build passed; 24 CTest entries: 23 passed, 1 failed

An inspectable technical study of public source code. No author affiliation, publication status, camera certification, or universal editor compatibility is asserted.

Contents

Abstract	4
Original English Abstract	5
Chapter 1. Research Object, Evidence, and the Scope of the Argument	6
Chapter 2. Signal Domains, Color Space, and the End-to-End Contract	8
Chapter 3. Deriving Inverse HLG and Reference Exposure	11
Chapter 4. Apple Log Encoding, Inverse Mapping, and Provable Properties	14
Chapter 5. YCbCr, Data Range, and Chroma Resampling	17
Chapter 6. The Creative-Adjustment Model, Proofs, and Limitations	19
Chapter 7. Floating-Point Precision, Error Budgets, and Numerical Validation	21
Chapter 8. The C++20 Core and External-Process Architecture	24
Chapter 9. The Triple-Buffer Pipeline and Memory Bounds	26
Chapter 10. Persistent CPU Workers and Determinism	29
Chapter 11. The CUDA Backend, Qualification, and Failure Atomicity	31
Chapter 12. Fixed-Frame-Rate Validation and Timeline Reconstruction	33
Chapter 13. Audiovisual Synchronization, Edit Lists, and Rotation Geometry	35
Chapter 14. MOV Atom Parsing and Semantic Comparison	38
Chapter 15. Apple Log Identification Fields and Metadata-Writing Proofs	40
Chapter 16. Metadata Allowlists, Creation Times, and Fidelity Semantics	42
Chapter 17. Tool Discovery, Execution Trust, and Numerical Qualification	44
Chapter 18. Process Supervision, Pipe Deadlocks, and Cancellation Correctness	46
Chapter 19. File Transactions, Disk Planning, and Crash Recovery	48
Chapter 20. GUI, CLI, Queues, and Inspectable Interaction	50
Chapter 21. Test Structure, the Present Reproduction, and Failure Analysis	52
Chapter 22. Performance Data, Complexity, and Reproducible Comparison	56
Chapter 23. An Implementation Walkthrough from One Sample to the Final File	59
Chapter 24. Discussion, Implementation Limits, and Research Conclusions	61
Appendix A. Symbols, Units, and a Review Guide	63
Appendix B. Repository Files and Corresponding Research Chapters	65
References and Source Locations	70

Edition and Evidence Notes

This English edition translates the existing Chinese paper without changing its research revision, conclusions, equations, figure sequence, or experimental records. The original Chinese PDF and materials are preserved. The full translated abstract and the original shorter English abstract are both retained to preserve the original structure.

The title uses mathematical proof to refer to the conditional derivations and structural arguments developed in the text. It does not assert machine-checked, whole-program verification. The reported tests belong to the original study; preparing this translation is not a new experimental run.

Mathematical curves are calculated from public constants and current source formulas. Architecture and state diagrams abstract implementation relationships. Figures labeled historical use repository version 1.2.0 measurements; figures describing the present reproduction use the actual September 29, 2026 study records. No illustrative curve is presented as an unreported experiment.

Equations, figures, and tables retain separate continuous numbering. Principal source files appear at the end of each chapter, linked to the fixed commit. Appendix B and the companion inventory cover every version-controlled file.

Reading route: Chapters 1-7 establish the signal and numerical model; Chapters 8-12 explain execution and clocks; Chapters 13-19 examine media semantics and reliability; Chapters 20-24 connect interfaces, reproduction results, performance, the implementation walkthrough, and conclusions.

Abstract

Designed for color interchange between mobile imaging and professional post-production, LogForge converts 10-bit BT.2020 HLG ProRes footage into Apple Log / BT.2020 ProRes 422 HQ. Although this may appear to be a change of video encoding, it involves interdependent questions of transfer-function interpretation, scene-exposure normalization, chroma-sampling phase, timeline reconstruction, container identification, and trusted execution of external tools. This paper studies the public repository at the fixed commit 6b43a93fbd4d1457ff20b4c99a030c0e398f520a, version 1.2.1, build 26927B. Source code, interfaces, tests, build configuration, the README, and versioned documentation provide the basis for an explanation extending from the signal model to the executable application.

The conversion is first decomposed into integer YCbCr decoding, nonlinear RGB reconstruction, inverse HLG opto-electronic transfer, reference-white normalization, optional creative adjustment, Apple Log encoding, and lossy output encoding and packaging. Piecewise functions, monotonicity, gray-card anchors, error propagation, and boundaries of invertibility are derived step by step. The analysis then examines the triple-buffer floating-point bridge, persistent CPU worker threads, and the CUDA implementation with runtime qualification. It explains why Standard mode can process arbitrary component blocks, whereas Creative mode must pair the complete RGB planes. Finally, timestamps, MOV atoms, audio copying, tool trust, failure recovery, and publication transactions are used to establish the conditions on which system correctness depends. Historical repository measurements and tests executed for this paper are reported separately.

The principal conclusion is conditional: when input interpretation, the numerical qualification of external tools, and output validation hold, LogForge implements traceable color re-encoding and a constrained media-delivery process. Mathematical derivation can explain conformity to the declared functional relationships; it cannot establish recovery of sensor information already lost, nor replace interoperability experiments with different cameras and editing applications. Version 1.2.1 differs from some historical documentation in input admission, defaults for missing chroma location, and pixel-rotation policy. These differences are explicitly resolved in favor of the current code. This paper is a reproducible source-code and engineering study; project statements, historical tests, and finite sampling are not elevated to unconditional certification.

Keywords: HLG; Apple Log; BT.2020; floating-point video pipeline; CUDA; MOV metadata; software reliability; reproducible verification.

Original English Abstract

LogForge is a native Windows application that converts BT.2020 HLG ProRes footage into Apple Log / BT.2020 ProRes 422 HQ. This monograph studies release 1.2.1 at an immutable repository revision. It derives the implemented signal transform, explains bounded CPU and CUDA processing, and examines timestamp reconstruction, metadata signaling, executable trust, cancellation, and transactional publication. Mathematical properties, repository measurements, and newly executed regression tests are distinguished throughout. The resulting argument is conditional: conformance to a declared encoding does not imply sensor-data recovery, camera-look equivalence, lossless transcoding, or universal editor compatibility.

Chapter 1. Research Object, Evidence, and the Scope of the Argument

1.1 From product requirements to research questions

In video post-production, standardizing material to a Log encoding entails at least three distinct objectives. First, every pixel must have a consistent signal interpretation so that the same technical input transform can be applied across footage. Second, editing software must read the gamut, transfer curve, and data range correctly, without manual import corrections for each file. Third, conversion must preserve frame count, audiovisual synchronization, orientation, timecode, and traceable capture information. Meeting only one objective does not constitute a complete media-delivery system. Writing an Apple Log identifier without changing pixels, for example, causes previously valid HLG data to be misinterpreted. Changing pixels while retaining HLG tags can instead trigger an additional, incorrect transfer-function conversion downstream.

LogForge is useful as a research subject because it assigns these objectives to explicit modules. The color kernel performs numerical mapping; FFmpeg performs decoding, matrix conversion, resampling, and ProRes encoding; the MOV analyzer and identification writer handle structured signaling; and the transcoding job supervises processes, validates results, and publishes files. This division allows implementation to be described as a sequence of contracts with inputs, outputs, and failure conditions, rather than as an opaque command line. The paper follows these contracts and does not substitute interface screenshots or subjective appearance for implementation evidence.

The research questions are therefore as follows. Given an input sample interpreted as BT.2020 NCL HLG and an explicit exposure convention, how is the corresponding Apple Log sample generated? When samples form a video file, how are intermediate precision and storage requirements controlled? When conversion spans multiple threads, external processes, and disk files, how is an error prevented from becoming an apparently complete final file? These questions concern the mathematical, execution, and delivery layers respectively. Their answers are related, but require different forms of evidence.

1.2 A fixed revision and repository-wide coverage

The local Git HEAD and remote origin HEAD were checked and both identified the fixed commit stated above. Recording a commit is preferable to recording only the main branch, because subsequent fixes cannot silently change the object being reviewed. Key implementations are located by file and function names. The appendix and accompanying manifests classify all version-controlled files and provide line counts and hashes. Project code, test scripts, build resources, and documentation are included in the analysis. The third-party nlohmann/json single header and generated PTX are recorded separately as a dependency and generated artifacts; their individual lines are not presented as algorithms independently designed by this project.

The repository contains documentation from several historical stages. The README is the user entry point, but the code at the selected commit is the final authority for implementation semantics. docs/ARCHITECTURE.md, docs/METADATA.md, and early validation records help explain design motivation, yet do not automatically override branches changed later. Current code, current release notes, and historical test records are compared in that order. Conflicts are identified explicitly rather than resolved by choosing the most favorable narrative. For example, older documentation requires an explicit declaration when chroma location is missing, whereas the current EffectiveChromaLocation returns left by default. This affects confidence in input interpretation.

1.3 Four non-interchangeable forms of evidence

This paper uses four forms of evidence. Source-code facts establish that a path exists, such as multiplication by exposure gain in the scene-linear domain. Mathematical derivation explains why a property holds under stated conditions, such as strict monotonicity of the creative luminance mapping for positive neutral inputs. Historical measurements establish that the repository recorded a result on a particular machine, driver, and version. The present reproduction establishes the build and test outcomes actually obtained in the current environment. Each form is useful, and each has boundaries that cannot legitimately be crossed.

A typical invalid inference is to move from agreement between CPU and GPU outputs on six input sets to permanent, exact agreement for every floating-point value. Another is to infer support in all players from successful identification in one Resolve version. Finite tests are used here to find counterexamples, build engineering confidence, and delimit compatibility. Genuine whole-domain claims require verifiable mathematical premises, language semantics, and implementation structure. Claims involving external encoders and the operating system retain the corresponding assumptions.

Table 1. Evidence classes and the conclusions they can support

Evidence	Example	Supports	Does not directly support
Source inspection	HLGToAppleLog call order	The current computational relationship	Exposure accuracy for arbitrary cameras
Mathematical derivation	A lower bound on the creative-curve derivative	Neutral-luminance monotonicity in the stated parameter domain	All color components remaining in range
Historical measurement	Version 1.2.0 performance JSON	Observations for the specified workload	A performance guarantee for version 1.2.1 on every platform
Present reproduction	Release build and CTest	Results in this environment	A new commercial-editor certification

1.4 Correctness as a conditional proposition

System success is expressed as a conjunction of predicates: the input is interpretable; the tools are executable and numerically qualified; conversion completes; temporal structure meets the requirements; metadata is consistent; and final publication succeeds. The engineering meaning of a successful return value must be understood through these conditions jointly. In particular, a valid output container must be distinguished from correct color: a structurally valid file may still contain severe color errors caused by an incorrect matrix, range, or transfer function.

$$S = I \wedge T \wedge C \wedge M \wedge V \wedge P$$

(1)

Here I denotes input and temporal conditions, T tool trust and qualification, C completion of color computation, M consistent media semantics, V successful validation, and P successful publication. This expression is not a single Boolean variable in the source code. It is a formal organization of the TranscodeJob lifecycle introduced by this paper. It makes explicit that the overall task should not be described as successfully converted if any one of these conditions fails.

Principal source files: [README.md](#); [CHANGELOG.md](#); [docs/RELEASE_1.2.1.md](#).

Chapter 2. Signal Domains, Color Space, and the End-to-End Contract

2.1 Five quantities that must remain distinct

To avoid using one symbol for both linear light and video code values, H denotes a normalized nonlinear HLG component; E denotes the relative scene quantity obtained through inverse HLG; R denotes the linear component after reference-white normalization and exposure; P denotes a normalized Apple Log component; and q denotes the final integer code value. Three-component vectors are written in bold. A color matrix acting on primed, nonlinear RGB must not be described as an operation on scene-linear luminance.

The BT.2020 primary coordinates and white point define color geometry. HLG and Apple Log define nonlinear relationships between components and linear quantities. The YCbCr matrix reorganizes three nonlinear RGB components into a luma-like component and color differences. Video range specifies the mapping from normalized values to integers. These are separate layers of convention. Describing a file as BT.2020 is insufficient to determine its display, because the transfer curve, range, sampling, and display transform remain unspecified.

2.2 Decomposing the end-to-end operation

In Standard mode, an input pixel follows this conceptual mapping. ProRes is decoded to recover 10-bit YCbCr. The values are normalized according to the input range, and three HLG-encoded RGB components are reconstructed using an explicit chroma phase. Each component then passes through the inverse HLG OETF, reference scaling, and exposure gain, followed by Apple Log encoding. After all pixels have been processed, the output path rematrixes, subsamples, and quantizes nonlinear Apple Log RGB before passing it to the ProRes HQ encoder.

$$\mathbf{P} = f(2^e S h^{-1}(\mathbf{H})) \quad (2)$$

The componentwise application of f and h is an essential condition. Standard mode does not depend on a neighborhood or require the other two channels of the same pixel. Planar data can therefore be divided into arbitrary blocks of complete floating-point samples. Optional Creative mode introduces A , a mapping dependent on RGB luminance, and all three components of a pixel must then be available together. Both modes end with the same Apple Log encoding function. Creative adjustment does not redefine the Apple Log constants.

$$\mathbf{P}_{\text{creative}} = f(A(2^e S h^{-1}(\mathbf{H}))) \quad (3)$$

Signal domains and production boundaries

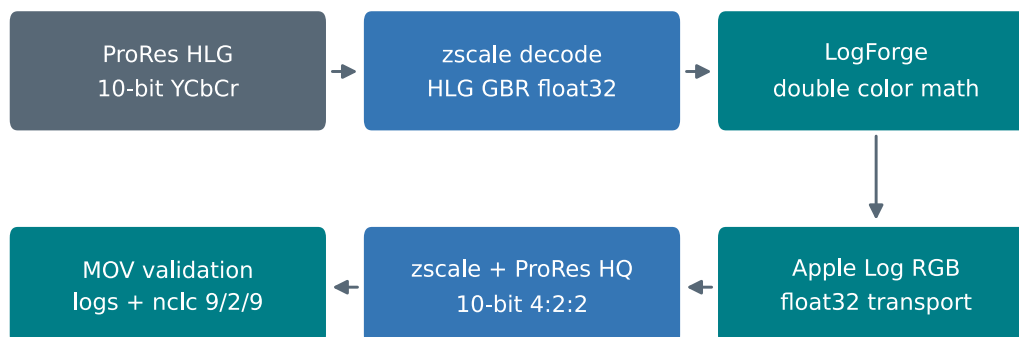


Figure 1. End-to-end data flow and signal domains. Blue denotes FFmpeg processing, green denotes LogForge numerical results or container processing, and gray denotes the source material. Follow the arrows to the right, down, and then left. The audio bypass is not expanded here. MOV signaling is corrected after encoding.

2.3 Why no gamut conversion is applied

Both input interpretation and output target use BT.2020 / D65. Changing a transfer curve does not require changing the primaries, so the program inserts no three-dimensional matrix from BT.2020 to ACES or another gamut. Apple's ACES IDT contains both Apple Log decoding and a conversion of primaries to ACES. LogForge uses the algebraic inverse of the former part. Copying the complete IDT would change the target gamut and depart from the project's stated purpose.

This distinction also explains why referring to Apple's ACES code does not make the output ACES. The paper must identify the mathematical object being reused and the color domain being retained. The relevant files are `Color.h`, `AppleLog.cpp`, and `HLG.cpp`. They introduce neither a hidden wide-gamut matrix nor a three-dimensional LUT. The BT.2020 primary declaration in the output must agree with this fact.

2.4 Scene interpretation and display interpretation

An HLG system includes not only an OETF but also an opto-optical relationship connecting scene and display. LogForge selects the inverse OETF to obtain a relative scene quantity. It does not calculate nits from display peak luminance or apply a display OOTF. This is a modeling choice determined by the conversion target: Apple Log remains an encoding for a post-production workflow, rather than a finished image delivered directly to a particular display.

A camera may nevertheless have applied local tone mapping, denoising, sharpening, and exposure processing before generating HLG. The inverse OETF can reverse the declared function, but not an unknown camera-processing chain. If a camera has already compressed distinct highlights into the same code value, even an exact inverse yields only the same linear estimate. E should therefore be understood as a relative scene quantity interpreted under this model, not automatically as an original sensor electron count or actual scene radiance.

2.5 Range and default interpretation in the source

The version 1.2.1 transcoder uses full range when the source explicitly reports pc, and limited range otherwise. The matrix and primaries are explicitly supplied to the filters under a BT.2020 NCL HLG interpretation. This behavior reduces rejection caused by missing tags, but places greater responsibility on the default interpretation. If a file actually uses another matrix and merely carries an HLG transfer tag, permissive admission cannot establish that it satisfies the intended assumptions.

This paper consequently separates format admission from reliable signal interpretation. The current `UnsupportedReasons` checks only the ProRes codec, Standard/HQ profile, and HLG transfer tag. Resolution, time base, duration, and related conditions are checked in later processing layers. The initial admission gate no longer enforces every historical restriction on color, sampling, or auxiliary metadata. Strict output validation does not eliminate the modeling risk that input defaults disagree with the actual recording configuration.

Principal source files: [src/Transcode.cpp](#); [src/Media.cpp](#); [docs/COLOR_PIPELINE.md](#).

Chapter 3. Deriving Inverse HLG and Reference Exposure

3.1 Inverting the piecewise OETF

HLG uses a square-root encoding in the low-luminance region and a logarithmic encoding in the high-luminance region. Let a be 0.17883277, with b and c derived from a as follows. Deriving these constants instead of entering independently rounded values reduces unnecessary disagreements between implementations. The source constructs b and c in each reference-function invocation. Whether a compiler folds those expressions is an optimization question and does not alter the function's definition.

$$a = 0.17883277, \quad b = 1 - 4a, \quad c = \frac{1}{2} - a \ln(4a) \quad (4)$$

For encoded components from zero to one-half, squaring the square-root branch and dividing by three gives E . Above one-half, subtract c , divide by a , take the natural exponential, add b , and divide by twelve. The two expressions correspond to the conditional branches in `DecodeToSceneLinear`. Natural and binary logarithms must not be interchanged: the high HLG branch uses the natural base, whereas the high Apple Log branch uses base two.

$$E(H) = \frac{H^2}{3}, \quad 0 \leq H \leq \frac{1}{2} \quad (5)$$

$$E(H) = \frac{\exp((H - c)/a) + b}{12}, \quad H > \frac{1}{2} \quad (6)$$

Checking the join is a simple implementation check. At H equal to one-half, the low branch gives one-twelfth. Substituting the definition of c into the high branch makes the exponential term equal to $4a$; using b equal to 1 minus $4a$ gives one-twelfth again. The derivatives can also be compared directly. The low-branch derivative is $2H/3$, while the high-branch derivative is the exponential term divided by $12a$. Both equal one-third at the join. This first-order smoothness follows from the construction of the constants and avoids an artificial corner at the branch boundary.

$$E' - (1/2) = \frac{1}{3} = E' + (1/2) \quad (7)$$

3.2 The engineering meaning of the negative extension

Matrix conversion and chroma reconstruction can produce small negative RGB components near edges. Immediately clipping them to zero changes the reconstruction filter's result and may introduce systematic errors in saturated colors or dark boundaries. HLG.cpp explicitly uses a negative-square extension for negative H . This is a preservation strategy for intermediate floating-point computation, not a claim that the nominal standard has been extended into camera-sensor ground truth.

$$E(H) = -\frac{H^2}{3}, \quad H < 0 \quad (8)$$

The extension is continuous at zero. Its negative-side derivative is $-2H/3$, so E increases as H rises from a more negative value toward zero. Unlike an unconditional square of H , the signed extension does not fold negative input into positive luminance. Its inverse is a negative square root, allowing consistent round-trip tests over the negative domain. Apple Log subsequently imposes a linear lower bound, however. Preserving negative values at this stage does not imply that the final encoding preserves their full negative amplitude.

3.3 Determining the reference-white scale

HLG's relative scene normalization is not itself the photographic reference quantity expressed as reflectance. LogForge adopts the BT.2408 nominal reference convention: a normalized HLG value of 75% corresponds to 100% reference white. Let inverse HLG at that point be E_w . Requiring R_w to equal one at zero exposure determines a unique scale factor S . This is not per-clip automatic exposure estimation, and the program does not infer a gray card from a histogram.

$$S = \frac{1}{E(0.75)} \approx 3.774118118465752 \quad (9)$$

$$R = S E(H) 2^e \quad (10)$$

The exposure parameter e is measured in stops. Increasing it by one multiplies the scene-linear quantity by two; decreasing it by one divides the quantity by two. Applying exposure in the linear domain preserves ratios among nonzero RGB channels in Standard mode. Adding the same constant directly to encoded HLG or Apple Log would produce a different nonlinear effect and would not constitute the same exposure model.

3.4 The numerical chain for gray and white cards

For an 18% neutral gray, R must equal 0.18 at zero exposure, so E equals 0.18 divided by S . This falls in HLG's low branch. Forward encoding gives H approximately 0.3782588831, and the Apple Log function then gives P approximately 0.4882724585. The example provides a complete sequence of coordinates from HLG through a linear quantity to Log. It is more useful for checking the program than the statement that gray should lie somewhere near the middle.

$$H_{18} = \sqrt{\frac{3 \cdot 0.18}{S}} \approx 0.3782588831 \quad (11)$$

A 90% white card corresponds to H approximately 0.7291933825, not 0.75. An earlier version anchored HLG 0.75 to 0.9, making the entire linear scale one-tenth lower than the current version. In exposure terms, this is log base two of 0.9, approximately -0.152003 stops. This difference illustrates the effect of reference calibration, but does not by itself explain arbitrarily large differences in the appearance of footage.

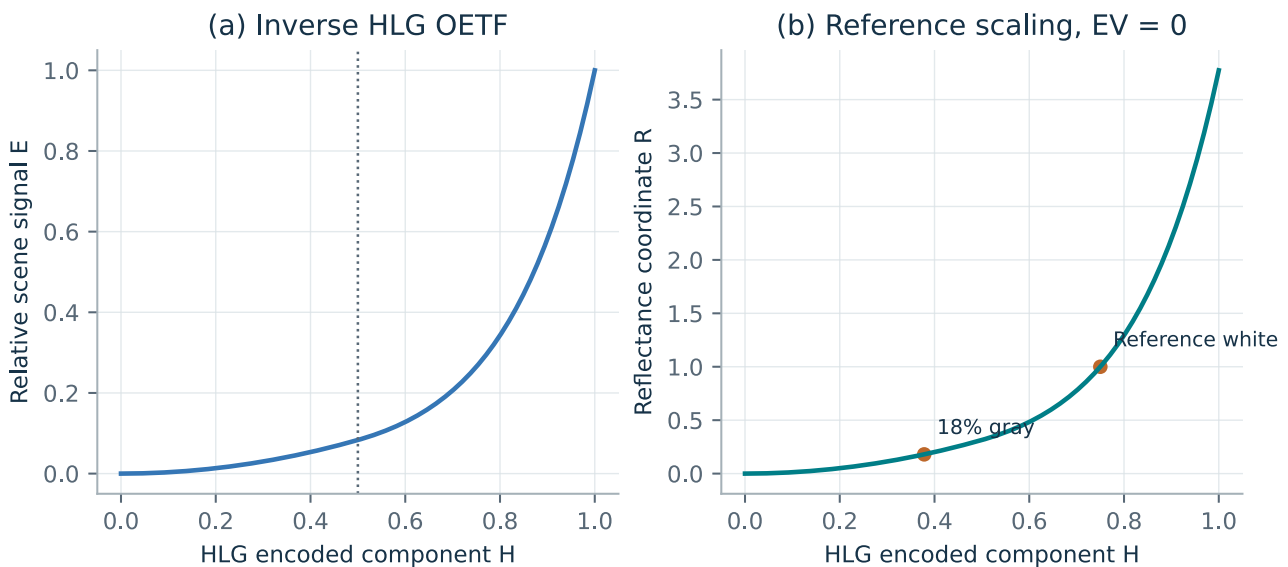


Figure 2. Inverse HLG and reference anchors. The left panel shows the relative scene quantity over the nominal input domain. The right panel shows reflectance coordinates after reference-white scaling. The marked values are model anchors, not measurements of camera exposure.

3.5 Gain, derivatives, and noise

Local error propagation can be described by a first-order approximation. A small perturbation of H changes the linear quantity by approximately S times the exposure gain times the derivative of inverse HLG. The derivative in the high branch increases with input, so equal encoded perturbations correspond to different linear perturbations. Conversion changes the representation of noise; it does not remove input quantization error.

$$\delta R \approx S 2^e E'(H) \delta H \quad (12)$$

This also explains the value of floating-point intermediates: they avoid imposing another narrow integer lattice between inversion and re-encoding. The original input is nevertheless only 10-bit. Floating point creates no new measurement precision; it merely allows existing discrete samples to pass through the formulas without additional low-precision truncation. The final 10-bit ProRes output is quantized again, so numerical precision must be considered across the entire chain.

Principal source files: [src/color/HLG.cpp](#); [include/logforge/Color.h](#).

Chapter 4. Apple Log Encoding, Inverse Mapping, and Provable Properties

4.1 Constants, thresholds, and three branches

Color.h centralizes the Apple Log constants, and AppleLog.cpp implements encoding and decoding. The linear floor R_0 is -0.05641088, the branch threshold R_t is 0.01, and the quadratic coefficient C is 47.28711236. The high branch uses $\beta = 0.00964052$, $\gamma = 0.08550479$, and $\delta = 0.69336945$. Centralizing these constants in a header allows CPU reference functions and tests to use the same public definition. The CUDA kernel still contains independent literals, so artifact hashes and numerical qualification remain necessary.

$$f(R) = \gamma \log_2(R + \beta) + \delta, \quad R \geq R_t \quad (13)$$

$$f(R) = C(R - R_0)^2, \quad R_0 \leq R < R_t \quad (14)$$

$$f(R) = 0, \quad R < R_0 \quad (15)$$

The branch order matters. The code checks the high branch first, then the quadratic branch, and otherwise returns zero. For ordinary finite inputs, this agrees with the mathematical definition. Rejection of non-finite inputs belongs to the outer TransformHLGToAppleLog layer. A low-level noexcept function accepting double must not be assumed to provide complete application-level guarantees for NaN and infinity.

4.2 Deriving the encoder from Apple's decoder

The high-branch decoder subtracts δ from P , divides by γ , raises two to that power, and subtracts β . Rearranging this relationship gives the logarithmic encoder above. The low-branch decoder takes a square root and adds R_0 ; solving for the encoded quantity gives the quadratic expression. This derivation shows that LogForge does not fit an empirical curve to a low-contrast appearance. It constructs a corresponding encoder by reference to a published decoding definition.

$$f^{-1}(P) = 2^{(P - \delta)/\gamma} - \beta, \quad P \geq P_t \quad (16)$$

$$f^{-1}(P) = \sqrt{P/C} + R_0, \quad 0 \leq P < P_t \quad (17)$$

$$P_t = C(R_t - R_0)^2 \quad (18)$$

In ideal real arithmetic, away from branch boundaries and with R at or above the floor, forward and inverse substitution cancels logarithm with exponential and square with square root, recovering the original quantity. This provides an algebraic basis for round-trip correctness. It holds only within the specified branches and domains. Every value below R_0 maps to zero, and the same zero cannot identify each distinct original value.

4.3 Join error from rounded constants

The published Apple Log constants are finite decimals. Evaluating the quadratic and logarithmic branches at R_t with these constants gives a very small difference. The current double-precision literals should therefore not be described as forming an absolutely exact continuous function. Nor should strict whole-domain monotonicity across the join be introduced as an unconditional theorem. The more precise statement is that each branch is monotonic internally, the join agrees within the specified numerical tolerance, and tests separately cover the neighborhood of the join.

In the logarithmic branch, the derivative is γ divided by the product of the natural logarithm of two and R plus β , and is positive. In the quadratic branch, the derivative is twice C times R minus R_0 , and is positive above the floor. Below the floor the function is constant, so the overall mapping includes a non-invertible plateau. The

curves in this paper are evaluated using the repository constants. The join error is invisible at ordinary plotting scales, but numerical validation remains necessary.

$$f'(R) = \frac{\gamma}{(R + \beta)\ln 2} \quad (R > R_t)$$

(19)

$$f'(R) = 2C(R - R_0) \quad (R_0 < R < R_t)$$

(20)

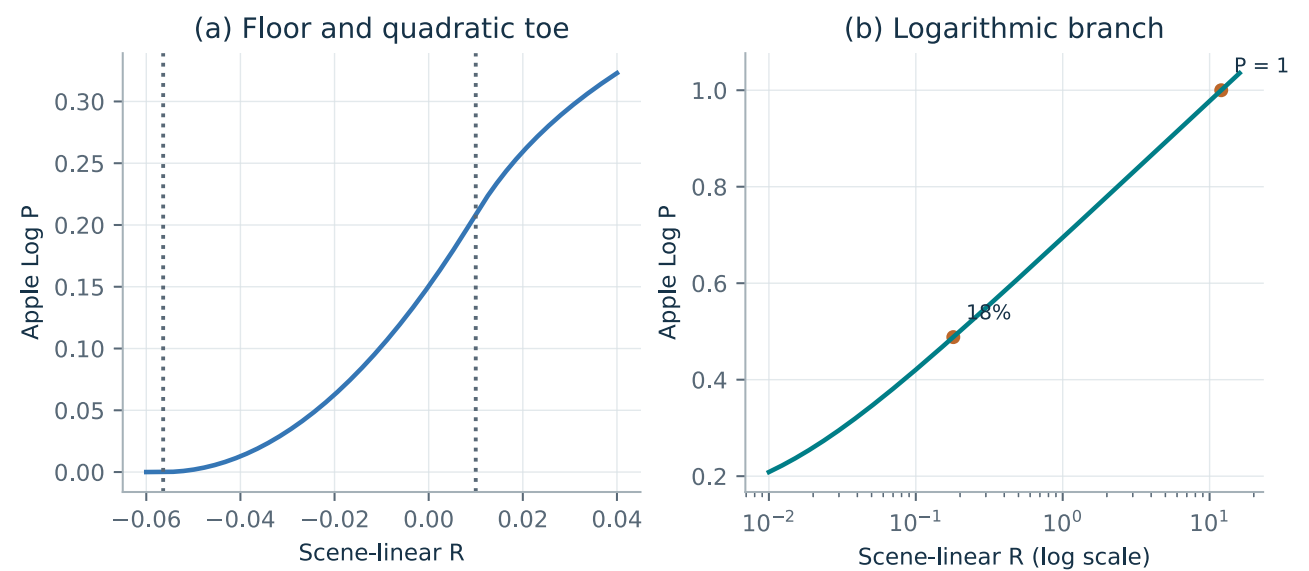


Figure 3. The quadratic shadow branch and logarithmic high branch of Apple Log. The floor, join, 18% gray, and nominal upper reference are marked. The curves are calculated from source-code constants and must not be interpreted as an experiment demonstrating dynamic-range recovery.

4.4 Interpreting black and nominal white

Linear zero does not encode to zero in Apple Log; it encodes to approximately 0.1504764523. If downstream processing treats that value as ordinary video black and stretches it back to zero, the Apple Log data has already been altered. An 18% gray encodes to approximately 0.4882724585, a 90% white to approximately 0.6816867959, and a linear value of twelve to approximately one. These are normalized function values, before mapping to limited-range YCbCr integers.

Table 2. Apple Log reference values and nominal video-range luma codes

Linear R	Normalized P	Neutral luma code 64 + 876P	Interpretation
0	0.1504764523	195.8173722	Encoded location of linear zero
0.01	0.2085553187	246.6944592	High-branch entry reference
0.18	0.4882724585	491.7266737	18% gray anchor
0.90	0.6816867959	661.1576332	90% white-card reference
12.0	0.9999999784	939.9999811	Nominal normalized upper reference

The table precedes rounding, dithering, and lossy ProRes compression. Each decoded pixel therefore cannot be required to equal the listed decimal exactly. A valid test must first specify sample locations, chroma boundaries, quantization rules, and error tolerance. Otherwise, a digit-by-digit comparison of finite-precision results with real-valued formulas produces failures with no engineering significance.

4.5 A rigorous counterexample to invertibility

Take two distinct linear values below R_0 . Their images under f are identical. By the definition of injectivity, no inverse over that range can recover the original input. Similarly, final 10-bit quantization maps a continuous interval to a finite set of integers. Distinct real values must therefore enter the same codeword. This is a limitation of sets and functions, independent of whether a human observer can see the difference.

$$R_a \neq R_b, R_a < R_0, R_b < R_0 \implies f(R_a) = f(R_b) = 0 \quad (21)$$

What the paper can establish is implementation of a specified mapping and validity of certain structural properties, not losslessness of the complete transcoding chain. Higher intermediate precision, prohibition of implicit tone mapping, and validation of encoding error are all important. None can undo the merging of information caused by earlier quantization or camera processing. Output assessment must therefore distinguish encoding correctness, editability, and the ability to recover information.

Principal source files: [src/color/AppleLog.cpp](#); [include/logforge/Color.h](#); [tests/tests.cpp](#).

Chapter 5. YCbCr, Data Range, and Chroma Resampling

5.1 Where the BT.2020 NCL matrix operates

The three planes of input ProRes are usually not RGB, but a luma-like component Y and two color-difference components. Y is a weighted combination of nonlinear R', G', and B', and cannot directly replace scene-linear luminance. The BT.2020 NCL weights are 0.2627, 0.6780, and 0.0593, summing exactly to one. When the three RGB components are equal, Y equals their common value and both color differences are zero. This is a basic invariant for testing the matrix.

$$Y = 0.2627R' + 0.6780G' + 0.0593B' \quad (22)$$

$$C_b = \frac{B' - Y}{1.8814}, \quad C_r = \frac{R' - Y}{1.4746} \quad (23)$$

The inverse first recovers red and blue from the color differences, then obtains green from the luma equation. FfmpegNumeric.cpp uses these equations directly to compute an independent reference, rather than asking the same filter to generate the correct answer. If production and reference paths both depend on the same incorrect configuration, agreement cannot reveal the error. Explicit integer patches and scalar expressions reduce this risk of circular validation.

$$R' = Y + 1.4746C_r, \quad B' = Y + 1.8814C_b \quad (24)$$

$$G' = \frac{Y - 0.2627R' - 0.0593B'}{0.6780} \quad (25)$$

5.2 Video range is not division by 1023

A 10-bit container offers codewords from zero through 1023, but nominal video range does not occupy that entire interval. Luma runs from 64 to 940, a span of 876. Chroma is centered at 512, with a nominal total span of 896. Correct normalization of video-range input must remove the offsets and use the respective scales. Dividing Y directly by 1023 lifts black and lowers white, after which the nonlinear functions further reshape the error.

$$Y = \frac{q_Y - 64}{876}, \quad C_b = \frac{q_{Cb} - 512}{896}, \quad C_r = \frac{q_{Cr} - 512}{896} \quad (26)$$

The reverse output mapping uses the same offsets and scales. Codes outside nominal range may carry super-white values or negative reconstruction headroom, but their eventual preservation depends on boundary behavior in the filters, quantizer, and encoder. The aboveNominalWhite counter means only that a normalized Apple Log component exceeds one; it does not establish that code-value clipping has occurred. Conversely, remaining below one does not prove that fine texture remains distinguishable after quantization.

$$q_Y^* = 64 + 876Y, \quad q_{Cb}^* = 512 + 896C_b, \quad q_{Cr}^* = 512 + 896C_r \quad (27)$$

5.3 The information structure of 4:2:2

For a frame of width W and height H, 4:2:2 contains WH luma samples and WH/2 samples for each color-difference component, totaling 2WH integers. When 10 significant bits are held in 16-bit storage elements, the raw planar data occupies 4WH bytes. In contrast, gbrpf32le has three complete planes with four bytes per sample, totaling 12WH bytes. Entering floating-point RGB therefore triples the data volume relative to this raw YCbCr representation.

The horizontal chroma resolution of 4:2:2 is half its luma resolution. Conversion to full-resolution RGB requires reconstruction filtering, while output requires subsampling again. These operations are not generally exact inverses, particularly at sharp color edges. Choosing Spline36 is an engineering choice of interpolation kernel, not

the creation of missing chroma detail. Color-correctness tests should avoid discontinuities when checking the matrix and use separate phase tests to examine sample positions.

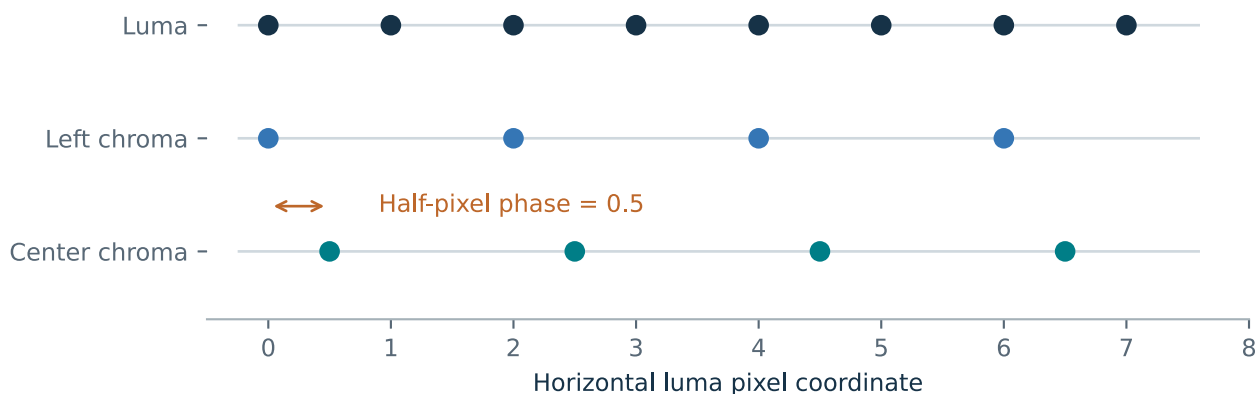


Figure 4. Spatial relationships among luma samples and left- or center-sited chroma samples. A half-pixel displacement produces a measurable difference on a chroma ramp. This is a sampling-geometry illustration, not a claim of recovered chroma resolution.

5.4 Why half-pixel phase requires a test

Uniform patches cannot distinguish left from center siting, because translating a constant signal leaves it constant. FFmpegNumeric uses a linear chroma ramp changing by eight chroma codes per horizontal luma pixel. A half-pixel phase difference then predicts a difference of four codes, far larger than the floating-point-stage tolerance. Sampling away from clamped endpoints reduces interference from boundary handling.

$$\Delta q_C = k \Delta x = 8 \times \frac{1}{2} = 4 \quad (28)$$

This experiment turns an apparently abstract filter option into an observable numerical difference. The input path tests left and center separately; the output path checks left siting through actual encoding and decoding. Output ffprobe may not report chroma_location, so the system retains explicit declarations and numerical evidence instead of inventing a nonexistent standard tag. A missing tag and incorrect pixel phase are different issues and must be recorded separately.

5.5 The purpose of equal transfer tags

On the decoder side, zscale receives arib-std-b67 for both input and output transfer. The intention is to perform matrix conversion, range conversion, and resampling without repeating an HLG display transform. On the output side, both transfer options are set to linear so that zscale bypasses transfer-function processing. The actual values at that point are already Apple Log. A subsequent setparams clears the output transfer declaration to unknown, after which container identification fields communicate Apple Log.

Here linear is a filter-configuration bypass, not a statement that Apple Log values represent linear light. Quoting the option without its context would be seriously misleading. Source comments and FFmpeg numerical qualification jointly establish the intent. Filter names and tags alone cannot establish that a future version retains the same behavior, so runtime testing remains part of the contract.

Principal source files: [src/FFmpegNumeric.cpp](#); [src/Transcode.cpp](#).

Chapter 6. The Creative-Adjustment Model, Proofs, and Limitations

6.1 Why creative processing has a separate switch

Standard conversion aims to implement the declared functional relationships faithfully, whereas creative processing deliberately changes scene-linear brightness and color. Separating them tells users why an output changes and tells tests which reference values must remain invariant. `ToneAdjustments` defaults to `enabled = false`. When disabled, the function returns the original RGB even if strong shadow-lift parameters have been saved. Tests compare the disabled state against the standard path sample by sample, ensuring that retained parameters cannot silently affect the result.

The enabled defaults are a three-stop shadow lift, one-stop highlight compression, and saturation 0.85. These are aesthetic choices made by the project, not values measured from a native Apple Log camera. The program writes the algorithm identifier `creative-luma-v1` and the effective parameters into metadata and the validation report, making it possible to trace whether a file contains only technical conversion or additional creative adjustment.

6.2 An exposure coordinate centered on middle gray

Luminance Y is first computed in scene-linear BT.2020. For positive Y , the base-two logarithm relative to 0.18 gives an exposure coordinate x in stops. Shadow and highlight controls can thus be defined by exposure distance, independently of input integer bit depth or Log curve. For nonpositive Y , the code selects $x = -6$ to avoid an invalid logarithm domain. That branch is an engineering convention requiring separate interpretation.

$$Y = \mathbf{w}^T \mathbf{R}, \quad \mathbf{w} = (0.2627, 0.6780, 0.0593)^T \quad (29)$$

$$x = \log_2(Y/0.18), \quad t = \min(|x|/6, 1), \quad w(t) = t^2(3 - 2t) \quad (30)$$

For negative x , the gain in stops is the shadow parameter times the weight. For nonnegative x , it is the negative highlight parameter times the weight. At six stops or more from middle gray, the weight saturates at one; near middle gray it falls smoothly to zero. The function examines only the current pixel, not neighborhoods or adjacent frames, and therefore does not automatically change its parameters when a histogram changes.

$$\Delta e = s w(t) \quad (x < 0), \quad \Delta e = -h w(t) \quad (x \geq 0) \quad (31)$$

$$g = 2^{\Delta e}, \quad \mathbf{R}_{\text{new}} = g [Y\mathbf{1} + \sigma(\mathbf{R} - Y\mathbf{1})] \quad (32)$$

6.3 The middle-gray fixed point and linear saturation

For a neutral 18% gray input, Y is 0.18, x and the weight are zero, gain is one, and the color-difference vector is zero. The output therefore remains the same gray. Saturation leaves every neutral gray unchanged because subtracting Y from each channel gives zero. For colored inputs, saturation scales the scene-linear color-difference vector, not differences between already encoded Apple Log channels.

Because the three luminance weights sum to one, the saturation part preserves luminance. Left-multiplying the bracketed expression by the weight vector cancels the weighted color-difference term and leaves Y . Applying the common gain then gives luminance gY . This proof assumes that no additional clipping occurs. If a channel is later clipped by a curve floor or integer boundary, the equality cannot be applied unconditionally in the output domain.

$$\mathbf{w}^T(\mathbf{R} - Y\mathbf{1}) = 0, \quad Y_{\text{new}} = gY \quad (33)$$

6.4 A derivative proof of neutral-luminance monotonicity

Let the output exposure coordinate z equal the input x plus the adjustment in stops. The smoothstep derivative is $6t$ times one minus t , with a maximum of 1.5 over zero to one. Since exposure distance is divided by six to obtain t , the maximum absolute derivative of the stop correction with respect to x is the control parameter divided by four. Both shadow and highlight parameters are bounded above by three. The derivative of z therefore has a lower bound of one-quarter.

$$w'(t) = 6t(1 - t) \leq \frac{3}{2} \quad (34)$$

$$\frac{dz}{dx} \geq 1 - \frac{3}{6} \cdot \frac{3}{2} = \frac{1}{4} > 0 \quad (35)$$

Thus, the creative curve cannot reverse the ordering of positive neutral luminances. Beyond six stops the weight is constant and the derivative returns to one. At middle gray and the six-stop boundaries the weight's first derivative is zero, giving a smooth join. The proof concerns this global curve in positive-luminance coordinates. It does not establish the ordering of arbitrary colored components or prove that all small differences survive as different quantized codes.

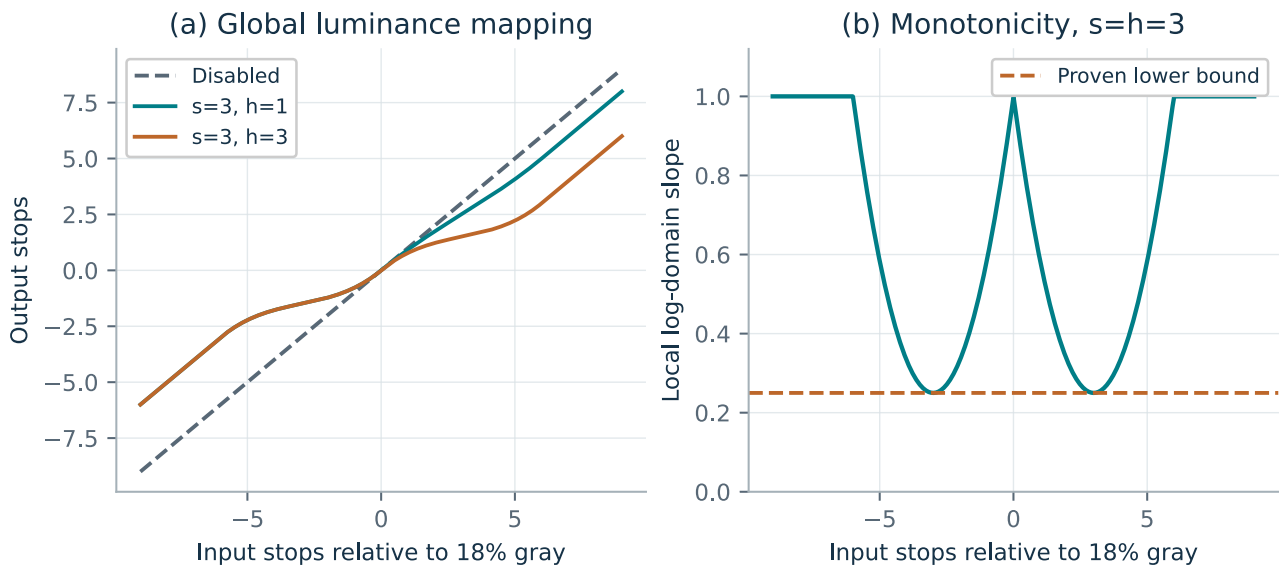


Figure 5. Creative luminance mapping and its derivative. The mapping panel compares disabled and enabled processing in exposure coordinates; the derivative panel shows the local slope at the parameter limits. The horizontal dashed line marks the theoretical lower bound of 0.25.

6.5 Saturation excursions and temporal stability

Saturation may reach 1.5, allowing the color-difference vector to be amplified. Even when the original RGB components are nonnegative, moving farther from the gray axis can make a component negative. At saturation one, a common gain preserves channel ratios. The luminance-centered saturation operation generally does not preserve the original ratios. Luminance monotonicity consequently cannot be rewritten as universal color safety or invariant hue.

Temporal stability follows from fixed parameters and pixelwise determinism: identical RGB input under identical parameters produces identical output, without storing the preceding frame's state. This avoids the pumping caused by changing scene statistics in adaptive-exposure systems. It does not remove noise, flicker, or capture-exposure discontinuities already present in the source. A deterministic function does not automatically smooth a temporal sequence; it simply avoids introducing additional content-driven parameter changes.

Principal source files: [src/color/HLG.cpp](#); [docs/CREATIVE_ADJUSTMENTS.md](#).

Chapter 7. Floating-Point Precision, Error Budgets, and Numerical Validation

7.1 Allocating precision along the pipeline

The program uses float32 for interprocess transport, promotes color expressions to double, and writes results back to float32. This balances bandwidth and computational stability. Using double in the pipes would double the transfer and buffer requirements of the three planes; using only low-precision approximations for exponentials and logarithms would make joins, shadows, and extreme exposures harder to reconcile with reference values. The current implementation does not replace the mathematical expressions with half precision, lookup-table interpolation, or fast approximate intrinsics.

Floating-point error cannot be dismissed with a single assertion that 10-bit precision is sufficient. Matrix conversion, function evaluation, float32 storage, rematrixing, quantization, chroma resampling, and ProRes compression each contribute error. Some contributions can be modeled as local perturbations; others include irreversible clipping. This paper uses a layered budget: scalar functions have their own tolerances, external matrix references have their own tolerances, and compressed integer codes have another set.

7.2 First-order sensitivity of the composite function

Writing the standard transform as $F(H)$, its derivative follows from the chain rule. Away from branch boundaries and clipping, the absolute derivative determines the first-order magnitude of a small input error propagated into P . Sensitivity differs between dark and bright regions, so a uniform input-code error does not remain uniform in the final Log domain. The illustration concerns mathematical sensitivity, not a measured noise model for a particular camera.

$$F'(H) = f'(S2^e E(H)) S2^e E'(H) \quad (36)$$

For a neutral patch, the output luma integer mapping introduces a factor of 876. A normalized error epsilon thus corresponds to approximately 876 epsilon luma codes. For example, a matrix tolerance of two times ten to the negative sixth power scales to approximately 0.001752 codes. This does not establish that the entire encoding chain has an error below that number, because ProRes and resampling contribute independently.

$$|\delta q_Y| \approx 876 |\delta P| \quad (37)$$

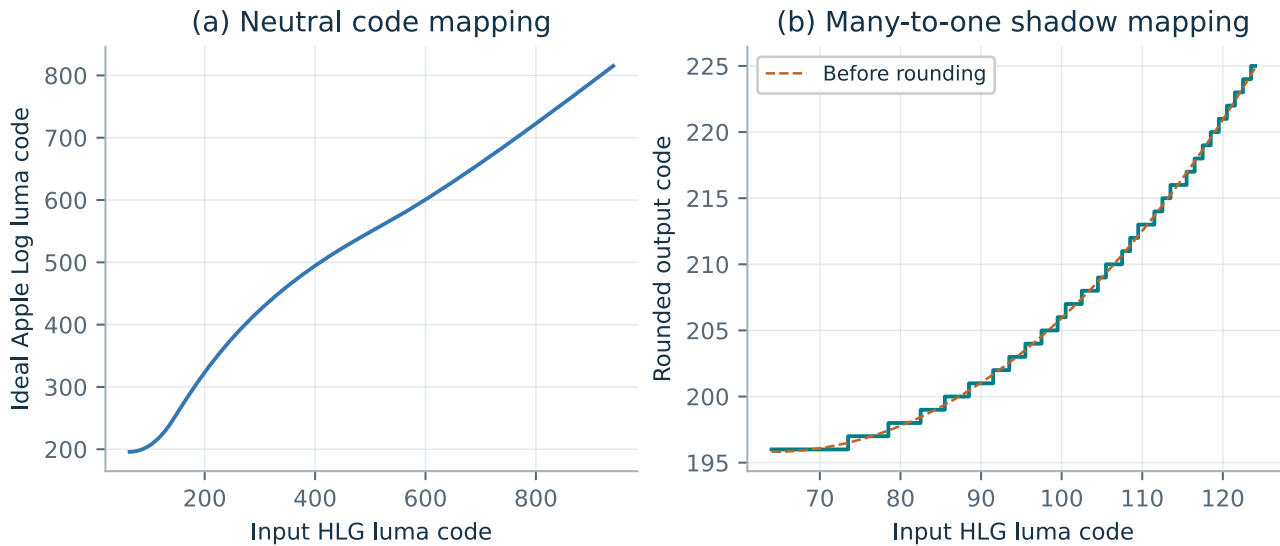


Figure 6. Quantization relationships after reference conversion of neutral 10-bit HLG codes. Several dark input levels may map to the same output integer, illustrating the non-invertibility of a finite-codeword mapping. The figure simulates rounding only and excludes ProRes compression.

7.3 Avoiding circular validation with reference signals

FFmpegNumeric constructs sixteen explicit integer YCbCr patches spanning near-black, middle gray, white, super-white, and color-difference variations. Explicit matrix expressions supply the reference decoding, followed by scalar color functions to calculate expected Apple Log values. On the output side, uniform reference float patches are generated independently to isolate effects from input-edge reconstruction. Matrix errors, range errors, and anomalies in the ProRes path can therefore be localized separately.

Input matrix and phase errors use a floating-point tolerance of two times ten to the negative sixth power. Uniform sampling points after an actual ProRes round trip use a two-code threshold. The independent scene tests in tests/signal_integrity.py use a separate criterion of less than three codes. These thresholds must not be merged into an arbitrarily enlarged allowance. A tolerance belongs to a specific fixture, observable, and statistical procedure, not to a general budget for concealing discrepancies.

7.4 Full-component counts versus sampled inspection

SignalStatistics accumulates input minimum and maximum, output minimum and maximum, lower-floor clipping counts, and above-nominal-white counts across all transformed components. A complete W by H by N video should contribute $3WHN$ components. This is a useful full-population structural invariant for detecting omitted work, duplicate accounting, or GPU commit errors.

$$N_{\text{samples}} = 3WHN_{\text{frames}} \quad (38)$$

These statistics are collected before final YCbCr quantization and ProRes encoding, however. They do not audit every decoded pixel in the final file. PixelSanity instead samples nine small blocks in the first, middle, and last frames, retains the input floating-point data, and later decodes the actual output to compare block means. The former covers all components at an earlier stage; the latter observes final encoding but is sparse in space and time. They complement one another and cannot stand in for each other.

7.5 Interpreting coarse pixel thresholds correctly

PixelSanity permits a peak block-mean error of thirty-two codes and an overall mean of eight codes. These limits are clearly wider than the two-code qualification test on uniform patches, because real texture, block boundaries, and lossy encoding affect small-block statistics. The purpose is to detect channel displacement, a seriously incorrect curve, or widespread numerical anomalies. It does not prove that every detail is accurate and cannot replace strict flat-signal testing.

An error confined to an unsampled frame or region may escape the sampling gate. Positive and negative errors within a block may also cancel, leaving a small mean difference. The paper therefore describes this module as a post-encoding gross-error check and retains its original scope characterization. Stronger validation could add frame and region coverage or use whole-frame difference metrics, at the cost of additional time, storage, and reference-alignment design.

Principal source files: [src/PixelSanity.cpp](#); [src/FFmpegNumeric.cpp](#); [tests/signal_integrity.py](#).

Chapter 8. The C++20 Core and External-Process Architecture

8.1 Module boundaries and dependency direction

LogForge uses C++20 and Win32. It does not link against FFmpeg's multimedia libraries, but launches separate `ffmpeg` and `ffprobe` processes and communicates through rawvideo pipes, JSON probe results, and textual progress protocols. This boundary reduces coupling between the main application and a particular multimedia ABI. It also makes error isolation, argument quoting, process cancellation, and executable trust responsibilities that the application must implement itself.

The core static library contains color, media, tool-management, platform, and transcoding logic. CLI and GUI link against the same core. The GUI displays information and collects options rather than implementing a second set of conversion mathematics. This dependency direction lets CLI tests cover the production task core and concentrates platform-specific details in `Platform.cpp`, `StorageSafety.cpp`, and the Win32 window modules. The relative independence of pure mathematical functions provides a clear boundary for a future platform port.

Application layers and orchestration relationships

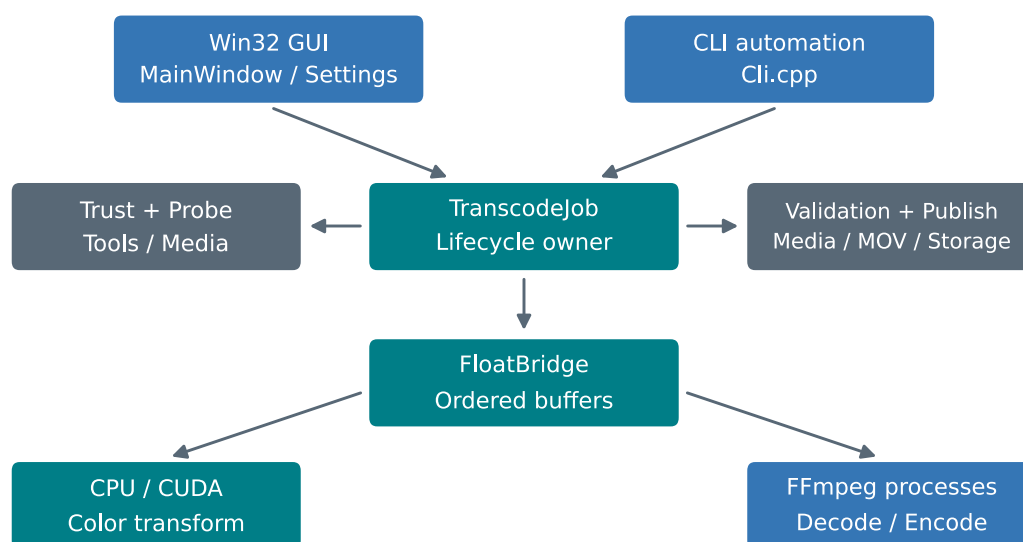


Figure 7. Source-module relationships. The GUI and CLI enter the same task core. Tool trust, temporal validation, color computation, and MOV structure handling provide distinct services. Arrows denote principal calls rather than every include relationship.

8.2 The transcoding job's sequence

`TranscodeJob` first reacquires a tool lease and verifies hashes, then checks numerical qualification, cancellation state, parameters, and input conditions. It next determines orientation handling, validates output paths and a writable log directory, analyzes the source MOV, checks the cadence of every video packet, and estimates disk requirements. Only after these prerequisites hold does it create a unique temporary output and initialize the backend and processes. This order exposes many failures before substantial transcoding resources are consumed.

During processing, the decoder produces floating-point samples, the bridge sends them to CPU or CUDA, and the encoder consumes the samples while copying audio. Completion is followed by checks of process exit codes, complete sample counts, orientation metadata, creation times, Apple Log identification, and output content. The validation report is saved before the final file is published. Progress callbacks span the stages, but an encoding progress value of 100% does not imply completion of final validation and publication.

8.3 The roles of the two FFmpeg processes

The decoder explicitly disables automatic rotation, selects the first video stream, removes audio, subtitles, and data, and outputs gbrpf32le. Necessary cardinal-orientation transforms are applied in the floating-point filter chain, avoiding an intermediate low-precision image format. The encoder uses the pipe as its video input and reopens the source media for audio and chapters. Video is encoded with ProRes HQ profile three, while audio uses stream copying.

Opening the input twice does not mean decoding all content twice: the second input primarily supplies audio and container information. The independent audio bypass also explains why changing video pixels does not require re-encoding audio. The clocks of the two inputs must nevertheless be aligned explicitly. If the original video starts at a nonzero time, the reconstructed pipe video and source audio would otherwise be displaced. The temporal policy is derived separately below.

8.4 Arguments and strings across boundaries

Platform.cpp assigns distinct roles to UTF-8 and UTF-16. JSON, logs, and most core strings use UTF-8; Win32 paths and process creation use wide characters. QuoteArgument handles quotation marks and backslashes according to Windows command-line rules, particularly the number of backslashes before a quotation mark or at the end of a string. Simply putting quotation marks around every path does not cover these cases.

CreateProcessW is called with an explicit executable path instead of relying on shell search. Arguments still require escaping for the target runtime's parsing rules, but the command text need not pass through an additional command interpreter. tests/tests.cpp covers spaces, embedded quotation marks, trailing backslashes, and Chinese paths. These apparently peripheral string rules are part of executing a video task correctly.

8.5 Exception boundaries and reports

Color exceptions, pipe exceptions, and validation failures propagate as AppError or standard exceptions. A reader thread cannot allow an exception to escape its entry point, since the process might terminate without cleanup. The implementation saves an exception_ptr, terminates the relevant processes, and rethrows on the main task. The GUI receives structured result messages. Logs retain the original external diagnostics instead of guessing at a translation of their meaning.

The validation JSON stores version, input and output probes, color parameters, signal statistics, tool identities, thread allocation, and stage timings. Each conversion thus has an inspectable evidence file. The report is not a formal proof certificate, but it lets a reproducer identify the assumptions used at runtime and distinguish parameter errors, external-tool errors, container errors, and publication failures.

Principal source files: [CMakeLists.txt](#); [src/Transcode.cpp](#); [src/Platform.cpp](#).

Chapter 9. The Triple-Buffer Pipeline and Memory Bounds

9.1 Why serial frame processing is insufficient

If the main thread reads a frame, computes the whole frame, and then writes it, color workers idle while decoding is awaited, the encoder lacks input while color computation runs, and pipe backpressure may block the decoder while the encoder consumes data. Even if every stage is internally multithreaded, gaps remain between stages. FloatBridge improves Standard mode by preparing three slots, allowing reading, computation, and writing to operate concurrently on different blocks.

Let the per-block service times for reading, computation, and writing be d , c , and e . Ideal serial processing of K blocks takes K times their sum. In a pipeline without resource contention, the slowest stage determines the interval after initial filling. Real systems also incur context switches, memory-bandwidth limits, and internal encoder buffering. The following expressions explain the mechanism rather than predicting exact performance.

$$T_{\text{serial}} \approx K(d + c + e) \quad (39)$$

$$T_{\text{pipe}} \approx d + c + e + (K - 1)\max(d, c, e) \quad (40)$$

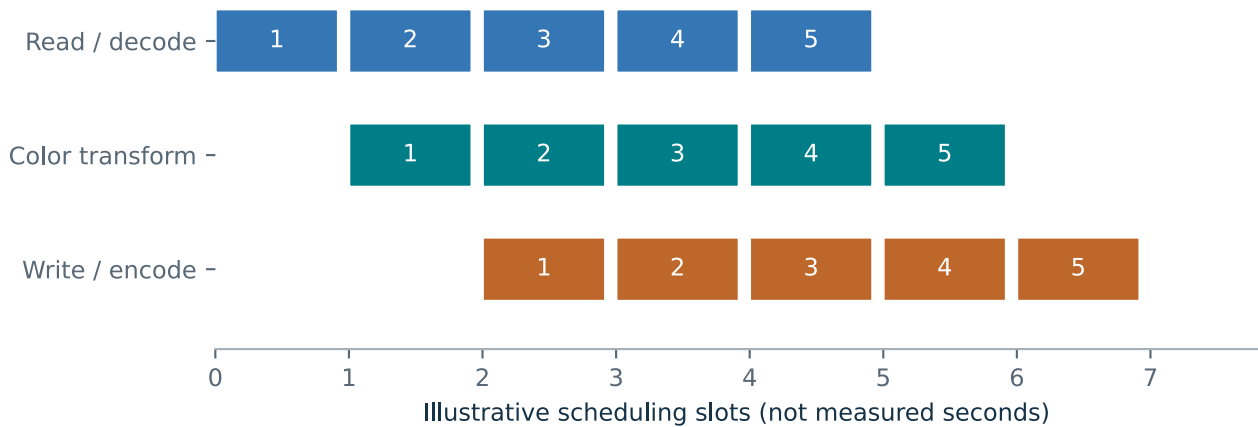


Figure 8. Conceptual timing of the three-stage pipeline. Color processing commits blocks in order, while reads and writes can overlap adjacent blocks. The intervals are instructional illustrations, not an actual Gantt chart extracted from performance logs.

9.2 The slot state machine

Each Slot contains data, filled, and state. State zero means writable; state one means decoded data is ready; state two means color transformation has completed and the writer may consume it. All three actors advance through the same circular slot order, preserving sample order under concurrency. Condition variables wait for available states, and a mutex synchronizes their publication and observation.

$$0 \xrightarrow{\text{read}} 1 \xrightarrow{\text{transform}} 2 \xrightarrow{\text{write}} 0 \quad (41)$$

The cycle also describes ownership transfer. The reader modifies data only in state zero, the color stage only in state one, and the writer reads data only in state two. In the absence of an unauthorized second accessor, two stages cannot write the same slot simultaneously. The implementation does not lock every floating-point sample. Exclusivity and visibility are controlled at block granularity, so synchronization cost depends on the number of blocks rather than growing at the same linear rate as pixel count.

9.3 End of stream and completeness

A single Read from the decoder pipe is not guaranteed to return all requested bytes, so the reader loops to fill its buffer. Standard mode permits a final short block, provided its byte count is a multiple of the size of float. Apart from a completely empty terminal block, Creative mode requires a complete frame on each read, ensuring that all three planes are present. A zero-length block travels through states one and two as the end-of-stream signal. On receiving it, the writer closes the encoder's standard input.

The total sample count must be divisible by $3WH$ before a complete frame count can be calculated. That count must then agree with the packet count checked earlier. These two checks detect a truncated byte stream and a mismatch between complete-frame count and media declarations, respectively. Waiting for ffmpeg to exit normally cannot replace sample accounting: an incorrect configuration can still generate a syntactically valid file with the wrong number of frames.

$$N_{\text{frames}} = \frac{N_{\text{samples}}}{3WH}, \quad N_{\text{samples}} \bmod (3WH) = 0 \quad (42)$$

9.4 Buffer size in Standard mode

The standard block limit is 1024 by 1024 floats, or 4 MiB. Three slots therefore require at most 12 MiB. For smaller frames, block capacity is the smaller of the frame's sample count and that limit. The color-bridge buffer consequently does not grow with clip duration and need not grow to a complete high-resolution frame. This bound applies only to FloatBridge slot data, not to the total memory of the process tree.

$$B_{\text{standard}} = 3 \times 4 \times \min(3WH, 2^{20}) \leq 12 \text{ MiB} \quad (43)$$

Decoders, encoders, pipes, thread stacks, sample records, and the CUDA driver consume additional memory. Cadence currently stores all packet times in a vector, which grows with frame count. Calling the entire application strictly constant-memory would therefore be inaccurate. The more precise conclusion is that the floating-point color bridge has a fixed block bound, while temporal analysis and external codecs have separate space requirements.

9.5 Why Creative mode retains a full frame

gbrpf32le stores a complete G plane, followed by a complete B plane and a complete R plane. For a pixel at linear index i , the three sample positions are i , $WH+i$, and $2WH+i$. Computing that pixel's luminance requires all three. Adjacent floats in a contiguous byte block are usually neighboring pixels in one plane, not three channels of the same pixel.

$$\text{GBR}(i) = [p_i, p_{WH+i}, p_{2WH+i}] \quad (44)$$

The current Creative implementation retains one complete planar frame and uses one bridge slot. This sacrifices the stage overlap of three slots but avoids pairing channels from different planes or frames incorrectly. A 4K frame requires 3840 times 2160 times 3 times 4 bytes, approximately 94.92 MiB. A more complex plane-reassembly buffer could be designed, but would require changing the protocol or adding per-channel storage. Merely shrinking the Standard-mode block size would not yield correct Creative processing.

9.6 Failure paths belong to the state machine

When any thread fails, fail saves the first exception, sets abort, wakes every waiter, and terminates the decoder and encoder. Every wait condition includes abort, so a thread cannot remain waiting for a state that will never arrive. During exception unwinding, JoinGuard terminates the external processes before joining the read and write threads, avoiding a thread stuck in an unfinishable pipe operation.

It is not enough to prove that states zero, one, and two cycle correctly during normal execution. Exit conditions must also break every type of wait. That is often the difficult part of reliable concurrent code. Library exceptions, user cancellation, broken pipes, and partial thread-creation failure must all converge on cleanup rather than leave a condition variable waiting for an owner that no longer exists.

Principal source files: [src/FloatBridge.cpp](#); [include/logforge/FloatBridge.h](#).

Chapter 10. Persistent CPU Workers and Determinism

10.1 Avoiding thread creation for every block

FloatTransformer creates a set of persistent jthreads in its constructor. Each Apply publishes a pixel span, increments an epoch, and wakes the workers. A worker remembers the epoch it has observed and starts processing only when a new task appears. The main thread waits for pending to reach zero before combining statistics. Numerous small blocks thus avoid repeatedly paying thread-creation and destruction costs, and thread lifetime remains distinct from the lifetime of an individual data block.

Work is allocated through an atomic next index with a Tile size of 16384. Each worker obtains the next non-overlapping region with `fetch_add`. The total count represents components in Standard mode and pixels in Creative mode. Because each position is processed by one claimant, sample operations have no cross-thread write conflict. A change in task-allocation order does not change the order of expressions within a sample.

10.2 Rearranging small RGB tiles in Creative mode

Creative mode copies corresponding positions from the three full-frame planes into a thread-private tile, still arranged as three compact planes. It calls the same scalar transform and writes the results back to their corresponding frame positions. Each worker therefore needs approximately three times Tile floats of workspace rather than a frame copy. For the final short tile, the actual length determines each plane's starting position, avoiding the mistake of treating the fixed Tile capacity as the valid span.

$$B_{\text{tile}} = 3 \times 16384 \times 4 = 196608 \text{ bytes} \quad (45)$$

The roughly 192 KiB creative scratch block per worker belongs in the memory accounting, but is much smaller than a full frame. The constructor allocates these blocks before starting threads, so allocation failure occurs within the caller's exception boundary. An uncaught resize in a thread entry point could instead terminate the process on memory exhaustion, preventing ordinary task-failure handling.

10.3 When bitwise agreement is possible

CPU parallel processing ultimately calls the same scalar function. The expression order for each component remains unchanged, and output does not depend on a sum across pixels. Standard output floats can therefore be compared bitwise with the scalar path. Creative mode likewise changes only pixel grouping, not the luminance and gain expressions within a pixel. The parallel checks in tests/hardening.cpp target precisely this structure.

Statistics reduction requires a separate explanation. Sample counts and risk counts use integer addition, which is order-independent within the non-overflowing range. Minima and maxima are taken over samples already checked for finiteness and likewise do not depend on merge order. There is no floating-point-mean associativity problem here because production signal statistics do not sum all pixels in floating point. If a future version adds means or variances, reduction order and reproducibility must be reconsidered.

10.4 Small tasks and thread budgets

For fewer than twice Tile samples, Apply directly uses the scalar path to avoid synchronization costing more than the work it saves. The default worker count is based on half the hardware concurrency and is capped. During transcoding, the more specific allocation comes from PlanThreads. That function allocates decoder, color, and encoder workers jointly, attempting to prevent each subsystem from independently occupying every logical CPU.

In the historical twenty-logical-CPU environment, the CPU backend assigned two decoder threads, nine color threads, and seven encoder threads, with additional filter threads. The CUDA backend gives more CPU budget to decoding and encoding while retaining a small color-worker allocation for Auto fallback. This budget is not the total operating-system thread count: FFmpeg, drivers, and runtimes may create helper threads. Thread tables must

state their counting scope to avoid treating configured worker counts and measured process-tree counts as contradictory.

10.5 The limits of CPU optimization

Persistent workers reduce scheduling overhead, but more threads do not guarantee greater speed. Small resolutions may be dominated by fixed costs; large resolutions may be limited by exponential evaluation, memory bandwidth, or the encoder. With several stages running concurrently, accelerating color computation may merely make a downstream stage the bottleneck. Correct optimization requires examining total task time rather than one function's runtime.

This is another reason to retain a scalar reference. An optimized implementation may change partitioning and scheduling, while a simple comparison path keeps performance changes separate from the color definition. Changing mathematical approximations, saturation behavior, or the placement of exposure to gain speed is no longer a pure implementation optimization. It requires a revised numerical contract.

Principal source files: [src/color/FloatTransformer.cpp](#); [tests/hardening.cpp](#).

Chapter 11. The CUDA Backend, Qualification, and Failure Atomicity

11.1 The dynamic driver interface and deployment boundary

CudaTransformer loads `nvcuda.dll` only from the system directory. It uses the CUDA Driver API to create a context, load PTX, allocate memory, launch kernels, and read events. Distribution does not require bundling `cudaart`, and an ordinary MSVC build does not compile CUDA source. The backend selects a device with compute capability at least 7.5 on which a context can be created. The embedded PTX targets `compute_75`.

This structure separates the development-time compiler from the runtime driver. Without a suitable driver, Auto can choose CPU, whereas forced CUDA reports an explicit error. The absence of a linked CUDA runtime DLL does not imply that GPU execution has no external dependencies: JIT compilation, device support, and driver loading remain necessary. Backend reports record device name, driver version, and qualification outcomes to help locate environmental differences.

11.2 Parallel organization of the kernel

ColorKernel uses a fixed grid of 256 blocks with 128 threads per block. Each thread traverses one or more components by a grid stride; Creative mode instead traverses pixels. Local statistics enter shared memory and are combined by a binary reduction into per-block statistics. The host finally merges the 256 records. Device-side pixel expressions still use double, with float input and output.

$$i = bT + t + kBT, \quad B = 256, \quad T = 128, \quad k \geq 0 \quad (46)$$

In Standard mode, a thread's output depends only on its current component, so grid-stride indexing gives coverage without overlap. In Creative mode, count is total sample count divided by three, and corresponding indices in all planes are constructed from the same i . Shared-memory reduction combines counts and extrema without requiring synchronization across blocks. The CPU performs the final cross-block merge after synchronization.

11.3 The meaning of precise compilation options

The generation script disables fused multiply-add and flush-to-zero and requests precise division and square root. It does not enable fast math. One purpose of disabling FMA is to reduce differences between CPU and GPU expression-rounding paths; disabling flush-to-zero preserves expected behavior for very small values. These options reduce approximation-related discrepancies, but do not theoretically guarantee bitwise agreement among mathematical libraries on different devices.

`tools/compile_cuda.py` writes PTX as a header array and records the CUDA source hash, PTX hash, compiler hash, version, and options in a manifest. `tests/cuda_artifact.py` recomputes the content hashes and checks the target architecture and critical compilation parameters. This prevents an edit to `.cu` from being mistaken for an update to the embedded PTX actually executed by the application. The existence of a source file does not mean that a released program already contains the behavior of that source revision.

11.4 Six qualification combinations

Backend initialization generates a three-plane fixture with 8192 samples per plane, covering neutral ramps, signed shadows, super-white values, saturated primaries, and fixed-seed random values. It runs both Standard and Creative modes at -8, 0, and +8 stops, giving six combinations. Each is compared with independent scalar CPU output, while sample count and both risk counts are also checked.

$$\max_i |P_i^{\text{GPU}} - P_i^{\text{CPU}}| \leq 2 \times 10^{-6} \quad (47)$$

A fixed seed makes the input sequence reproducible and useful for cross-driver investigations. Extreme exposures reach nonlinear boundaries and range risks more readily than zero-exposure tests alone. The backend enters production only after all six combinations pass, which differs materially from using a GPU merely because an NVIDIA adapter was detected. Coverage remains finite, however. A measured maximum error of zero on these samples does not establish strict bitwise agreement for every possible input.

11.5 The commit point for host data

Apply first copies the original block into separate pinned host memory, then asynchronously uploads it, runs the kernel, and downloads results and statistics. After synchronization it checks finiteness, sample counts, and event results. Only then does memcpy overwrite the caller's samples and merge cumulative statistics. The original samples and cumulative statistics remain unchanged until every GPU operation succeeds. This is the basis of safe Auto fallback.

CUDA transaction for one host chunk

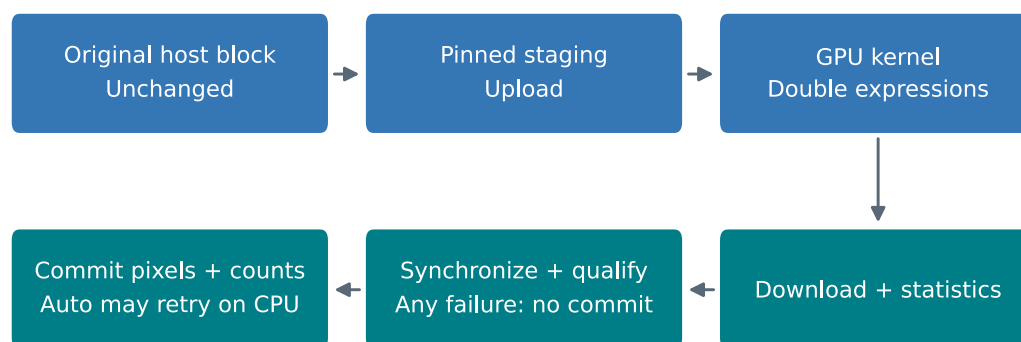


Figure 9. CUDA block execution and its commit boundary. Caller data is overwritten only after synchronization, statistics, and error checks succeed. On failure, Auto can submit the original block to CPU; forced CUDA terminates the task.

This property can be expressed as block-level failure atomicity. An exception before the commit point leaves host input equal to the input at entry to Apply. CPU retry therefore receives untransformed data rather than applying inverse HLG again to already encoded Apple Log. Downloading directly over the original input before checking errors would allow partially completed data to be retried, risking a locally doubled transform.

11.6 The actual costs of CUDA acceleration

GPU time includes upload, kernel execution, and download, with additional pinned-memory copies, context creation, qualification, and CPU codec work. Fixed costs are proportionally large for short clips, so a fast kernel may yield only limited end-to-end improvement. Creative mode needs full-frame pinned and device buffers, giving it substantially greater memory requirements than CPU Standard mode.

The source uses one nonblocking stream and synchronizes commits block by block. It is not an indefinitely deep, multistream, all-GPU video pipeline. ProRes decoding and encoding remain on CPU, as do zscale matrix and range operations. Describing the project as full-pipeline GPU hardware transcoding would be inaccurate. It accelerates the color-mathematics stage and uses thread budgeting and bridging to reduce waiting elsewhere.

Principal source files: [src/color/CudaTransformer.cpp](#); [src/color/ColorKernel.cu](#); [tools/compile_cuda.py](#); [tests/cuda_artifact.py](#).

Chapter 12. Fixed-Frame-Rate Validation and Timeline Reconstruction

12.1 Information lost by rawvideo

A rawvideo pipe carries sample bytes, not each frame's original timestamp, container edit lists, or time-base descriptions. The encoder must reconstruct video time from a declared frame rate. Feeding arbitrary variable-frame-rate input directly into this pipe would rebuild frame intervals uniformly, potentially changing motion speed and audiovisual synchronization. Fixed-cadence validation is therefore a prerequisite of this architecture, not an optional hint.

MediaInfo retains average and nominal frame rates, but Cadence does not make its final decision by checking whether they are equal. Nominal rate may be a probe estimate, while the average can be affected by container duration and quantization. The values that actually require inspection are the PTS and duration of every video packet. This explains why the project can accept a nonstandard but fixed cadence such as 29.99 while rejecting a drifting sequence whose average appears normal.

12.2 Describing cadence in integer time-base units

Let the time base be τ seconds per tick, packet i begin at integer timestamp p_i , its duration be d_i , and a candidate period be T ticks. Fixed cadence requires every duration to be near T , adjacent starts to be separated by approximately T , and accumulated phase to be near iT . The program also strictly requires each adjacent PTS difference to equal the preceding packet's declared duration, first excluding gaps or overlaps in the internal time table.

$$\varepsilon_d(i) = |d_i - T|, \quad \varepsilon_i(i) = |(p_i - p_{i-1}) - T| \quad (48)$$

$$\varepsilon_\phi(i) = |(p_i - p_0) - iT| \quad (49)$$

All three error limits are 1.05 ticks, in the input video time base, not milliseconds or hundredths of a frame. Expressing the threshold in milliseconds without `time_base` loses its meaning. Repeated or decreasing PTS, missing or nonpositive duration, and disagreement between an interval and the preceding duration all fail early, with a zero-based packet index recorded.

12.3 Exact and candidate periods

If every packet duration and adjacent interval is exactly equal, that integer period is authoritative. With time base n/d and packet period m , the frame rate is $d/(nm)$, reduced by the greatest common divisor. The code checks representable range before multiplication to prevent overflow in the product of time base and a long period. This branch does not force 29.99 to 30000/1001 merely because both are close to 30.

$$f = \frac{1}{m\tau} = \frac{d}{nm} \quad (50)$$

When clock quantization makes periods unequal, the program first uses the most frequent packet duration as a conservative anchor. If the period implied by nominal frame rate lies within the observed duration range, it becomes the candidate. Errors are then checked across every packet. A fitted average is not used to remove accumulated drift, because such fitting could disguise an actual cadence change as normal behavior.

12.4 Why accumulated phase must be checked

Consider a very small excess in each local frame interval. Every individual error may remain within tolerance, yet after hundreds of frames the start-time phase error can exceed a frame. Local-interval checks alone miss this persistent drift. Accumulated phase directly compares the ideal clock measured from the first frame with the actual clock, eventually causing a systematic offset to exceed the threshold and be rejected.

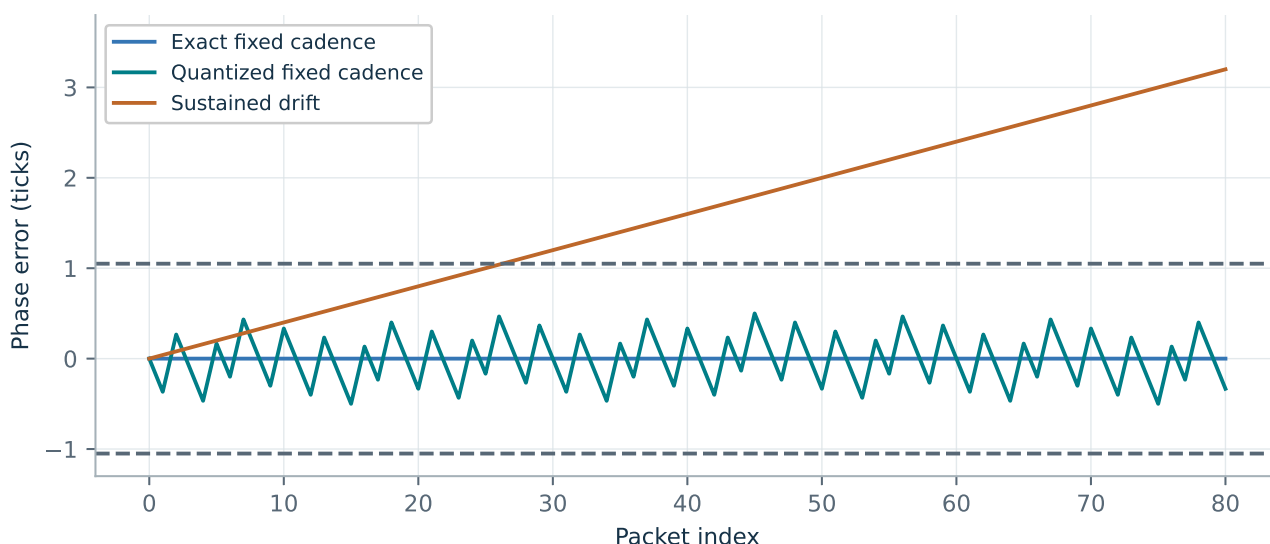


Figure 10. Phase examples for exact fixed cadence, quantized alternating cadence, and sustained drift. Dashed lines show the 1.05-tick budget. These instructional curves demonstrate that small local error does not guarantee small accumulated phase error.

Conversely, a correct rational frame rate on a coarse time base may require alternating long and short integer durations. If accumulated phase remains within budget, such quantization jitter should not simply be equated with arbitrary VFR. The policy allows bounded quantization around an interpretable fixed clock; it does not preserve every original timestamp. Accordingly, the paper describes validated reconstruction of fixed cadence rather than completely lossless copying of packet timestamps.

12.5 Why the output must be checked again

Passing input validation establishes a basis for pipe reconstruction, not proof that the encoder actually produced the expected cadence. Output validation traverses every video packet again and compares frame count, average frame rate, validated frame rate, and video duration. This can reveal changes caused by incorrect commands, encoder synchronization policy, or the muxer's time base. Media.cpp explicitly contains floating-point frame-rate thresholds and duration tolerances. They should be interpreted as implemented, rather than replaced by a claim of exact integer equality everywhere.

Temporal scanning stores all packets in a vector, giving linear storage in packet count N . Building the duration-frequency table additionally involves ordered-map operations. Costs are manageable when footage contains only a few distinct duration values, but very long clips still increase preliminary scan time and memory. Future streaming, multipass validation could reduce memory, but would have to retain equivalent candidate-selection and phase-checking semantics.

Principal source files: [src/Cadence.cpp](#); [src/Media.cpp](#).

Chapter 13. Audiovisual Synchronization, Edit Lists, and Rotation Geometry

13.1 Audio copying does not automatically ensure synchronization

Audio stream copying avoids another lossy encoding pass and preserves the original payload, but does not automatically guarantee the correct relative position after muxing. Source video may start at a nonzero time, while audio may include leading delay or edits corresponding to encoder priming. Pipe video is reconstructed from a new origin, so audio time must be interpreted relative to the same origin. The current encoding command uses `copyts` and applies the negative video `startTime` as an offset to the source input.

If source video begins at v_0 and audio at a_0 , the target relative difference is a_0 minus v_0 . Translating all input timestamps does not change this difference. `Media.cpp` directly compares input and output audio start relative to video start, with a 0.05-second tolerance. It separately checks duration differences for tracks with duration information and strictly compares codec, sample rate, channel count, and layout.

$$\Delta_{av} = a_0 - v_0, \quad |\Delta_{av}^{\text{out}} - \Delta_{av}^{\text{in}}| < 0.05 \text{ s} \quad (51)$$

13.2 Why movie timescale is preserved

A MOV movie time base, media time bases, and individual audio-track time bases may differ. A short audio offset represented in integer ticks may be quantized if the movie timescale is replaced by a coarser one. `InspectMovTimeline` reads a valid timescale from `mvhd`, and the transcoder passes it to the output muxer. The purpose is to preserve temporal representational capacity, not to prove byte-for-byte copying of every field.

The repository's Blackmagic regression uses a start offset of 500/48000 seconds, approximately 10.4167 milliseconds. This is substantially below 0.05 seconds, so the generic output threshold alone cannot establish exact preservation. The dedicated regression additionally checks integer timestamps and hashes of audio payloads. Generic production validation and targeted regression have different responsibilities and are described separately here.

$$\frac{500}{48000} = \frac{1}{96} \text{ s} \approx 10.4167 \text{ ms} \quad (52)$$

13.3 Edit-list handling in version 1.2.1

Edit lists map movie time to media time and can express leading empty segments, trimming, and temporal placement. Earlier versions attempted to reject unfamiliar lists through particular patterns. Version 1.2.1 instead records `elst` and lets FFmpeg interpret actual playback time. Video and audio entries are marked as interpreted by FFmpeg, other auxiliary streams are omitted, and existing timecode is regenerated under the safe policy.

This change resolves the case in which an iPhone `mebx` metadata track with a nonzero media origin blocked an otherwise normal video conversion. It should not be described as a complete implementation of all edit-list semantics by LogForge. The selected FFmpeg still performs the actual decoding and copying. LogForge retains checks on packet cadence, audiovisual relative starts, and output duration, constraining the task through these results instead of using the old camera-specific shape allowlist as an admission gate.

13.4 Display matrices and angle tags

A QuickTime display matrix can contain translation, scaling, and perspective, not merely an angle. SupportedDisplayMatrix parses nine integers and checks exact arrangements for the four permitted unit rotations. If translation is present, it verifies that it moves the rotated raster bounds precisely back to the origin. The ordinary linear part uses 16.16 fixed point, whereas the final homogeneous scale uses 2.30. Dividing every integer by the same number would be incorrect.

Under the row-vector convention used by the code, the coordinate relationships are as follows. The minimum coordinates of the four corners are negated to obtain the origin-correction translation. A rotation angle reported by ffproub alone cannot establish that a matrix containing other transformations is equivalent to a pure rotation. Current pixel baking therefore applies only to orientation paths that pass SupportedDisplayMatrix and correspond to multiples of ninety degrees.

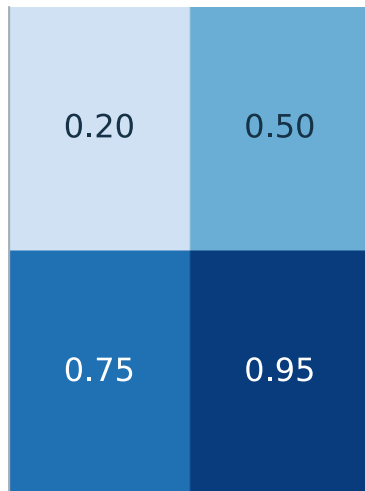
$$x' = ax + cy + t_x, \quad y' = bx + dy + t_y \quad (53)$$

$$t_x = -\min(0, aW) - \min(0, cH), \quad t_y = -\min(0, bW) - \min(0, dH) \quad (54)$$

13.5 Baking orientation in the floating-point domain

For ninety- and 270-degree rotations, the program swaps output width and height and inserts transpose at the gbrpf32le stage of the decoder filter chain. A 180-degree rotation uses horizontal and vertical flips. The operation permutes sample positions without interpolated scaling, so it introduces no additional resampled colors. A 3840 by 2160 source carrying portrait orientation can become an actual 2160 by 3840 upright raster with an identity final display matrix.

Source raster + rotation metadata



Upright pixels + identity matrix

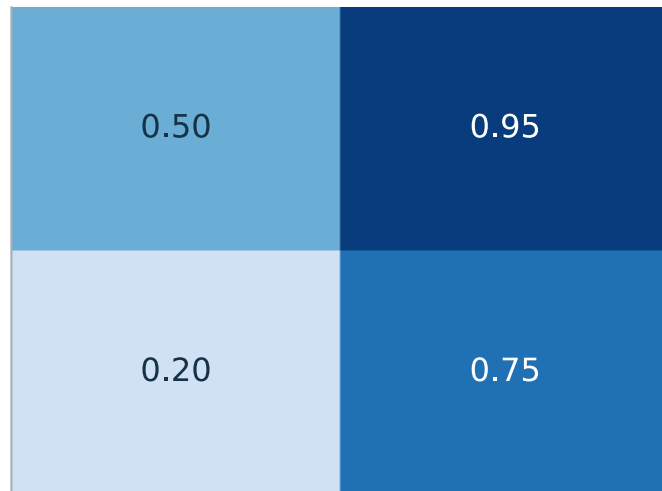


Figure 11. Pixel rotation illustrated with a four-quadrant fixture. A 90-degree path exchanges width and height and permutes pixels. The identity output matrix removes the player's need to interpret an orientation tag. The operation is geometric rather than a new color transform.

For a pure permutation, a neighborhood-independent pixelwise color transform commutes with rotation: rotating before computation and computing before rotation give equal values at corresponding pixels. This explains why rotation can precede the floating-point color transform. The real application also performs YCbCr chroma resampling. The commutation claim applies only to reconstructed RGB and the pixelwise color kernel, not to an arbitrary complete pipeline containing chroma filters.

$$F(\Pi\mathbf{x}) = \Pi F(\mathbf{x}) \quad (55)$$

13.6 Remaining boundaries of the orientation policy

When material does not meet the standard matrix conditions, the current version does not arbitrarily bake a reported angle into pixels; it may still use metadata-preservation and remuxing paths. The older statement that unsupported matrices are rejected cannot simply be transferred to current admission code. Nor can complex matrices be described as fully rasterized. Final validation principally checks rotation angle and header restoration. Complex display semantics still require dedicated footage and editor tests.

Audio layouts follow a related principle: two unlabeled channels are not necessarily standard left and right. The program uses `guess_layout_max 0` to prohibit inference and requires the output layout string to match the input. Preserving an unknown value can be more honest than writing a plausible default. Compared with defaulting missing chroma location to left, this also shows that default policies differ across fields and should be presented clearly at the interface.

Principal source files: [src/MediaSafety.cpp](#); [src/Transcode.cpp](#); [tests/compatibility_v121.py](#).

Chapter 14. MOV Atom Parsing and Semantic Comparison

14.1 Why an independent container analyzer is needed

ffprobe exposes many media fields, but does not expose every sample-description extension and private identifier in the same way. To investigate native Apple Log identification, the project implements ReferenceMovAnalyzer, which reads hierarchical MOV atoms directly. It does not decode pixels. It examines sizes, hierarchy, typed metadata, color extensions, creation times, and display matrices, filling gaps in generic probe output.

Binary-container analysis begins with boundaries. An ordinary atom has an eight-byte header; size one selects an extended 64-bit size, and size zero extends to the current container's end. The parser must ensure that sufficient bytes remain, the length is no smaller than the header, and the length does not exceed the parent boundary. Treating size as trusted could turn a damaged file into an out-of-bounds read, a huge allocation, or an infinite loop.

$$8 \leq L \leq E - O, \quad O_{\text{next}} = O + L \quad (56)$$

Here O is the current offset, E the parent-container end, and L the validated atom length; an extended header additionally requires sixteen bytes. The code prefers comparisons of length against remaining space to avoid first performing a potentially overflowing offset addition. Structural validation is independent of pixel correctness, but determines whether metadata operations can locate their targets safely.

14.2 Bounded parsing instead of loading the whole file

Reader limits a single metadata read to 4 MiB, atom count to 100,000, and nesting depth to twenty, and also limits key and edit-list counts. mdat is not treated as metadata requiring complete loading, so a long video's media payload does not directly enter parser memory. Small unknown extensions may retain hexadecimal bytes; large ones record only size, avoiding unbounded copying.

These limits are defensive policies and may reject valid but unusually large metadata structures. Safety is therefore not equivalent to accepting every MOV file. For a small desktop transcoder, rejecting structures that cannot be interpreted reliably is easier to maintain than an unbounded attempt. The limits must still be identified as project choices rather than universal maxima imposed by the MOV standard.

14.3 Resolving keys and ilst indices

Modern QuickTime metadata often centralizes key names in keys. Integer identifiers in ilst refer to those indices, with actual values carried by data atoms. The parser first scans siblings for keys and then processes ilst, allowing correct key resolution even when ilst precedes keys in the file. A parser interpreting strictly in file order could mistake an early index for a FourCC.

The beginning of data specifies type, flags, and locale, followed by the value. The implementation distinguishes UTF-8, UTF-16, signed and unsigned integers, floating point, and unknown binary data. Floats must be finite, and strings must satisfy the relevant encoding conditions. Printable bytes are not automatically valid text, and the name of an Apple API constant is not itself a verified serialized file format.

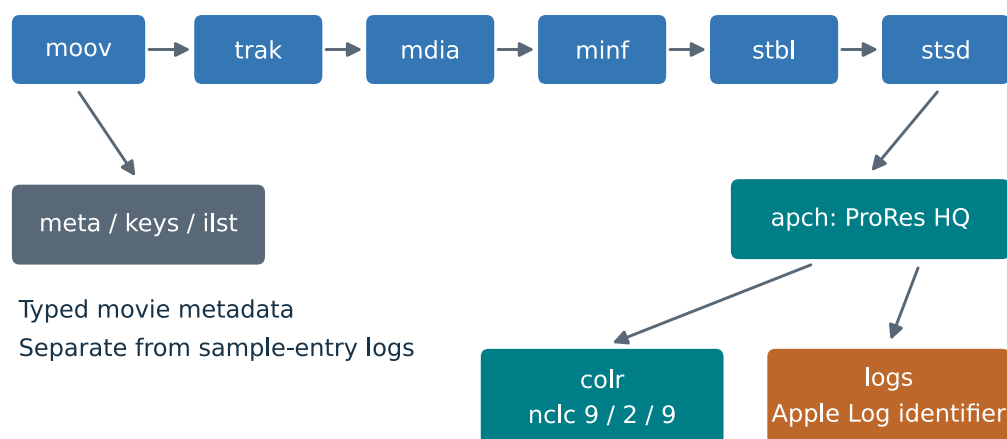


Figure 12. MOV hierarchy relevant to Apple Log identification. logs and colr are direct children of the ProRes sample entry. Movie-level meta/keys/ilst supplies a separate key-value metadata system; the two locations must not be conflated.

14.4 Special handling of the four-byte terminator

A real QuickTime video sample description may end its extension list with exactly four zero bytes. This is not a complete atom header and must not excuse missing bytes at the end of arbitrary containers. Reader accepts it only in the video-extension context when exactly four bytes remain and all are zero, recording the position in `video_sample_terminators`.

This detail illustrates how reference-file research informs a parser. Excessive strictness may reject a valid native structure; excessive permissiveness may accept truncation as normal. The correct adjustment is a special case with explicit context and length conditions, not global disregard for every tail shorter than eight bytes. The identification writer later uses the recorded terminator position to insert a new extension before it.

14.5 Semantic differences versus binary differences

Two MOV files may differ extensively in bytes because of atom ordering, encoded sizes, or media offsets while still sharing the same color interpretation. `SemanticDiff` removes offsets, sizes, and similar fields unsuitable for direct comparison from sample entries and extensions, collects parsed metadata, and combines it with `ffprobe` media summaries to produce a JSON difference. It asks which interpretation-relevant fields differ, rather than claiming binary identity.

Semantic normalization may nevertheless omit the effects of fields that are not understood. Unknown private bytes remain raw evidence; the absence of a parser name does not prove irrelevance. Controlled A/B imports into an editor isolate variables one at a time before connecting the logs identifier with automatic recognition. The analyzer provides candidate differences; causal conclusions still depend on experimental design.

Principal source files: [src/MovAnalyzer.cpp](#); [tests/hardening.cpp](#).

Chapter 15. Apple Log Identification Fields and Metadata-Writing Proofs

15.1 Three declarations with different responsibilities

The output contains a standard color description, project-specific declarations, and an identification extension established from native references. `colr` uses `nclc` values 9/2/9: BT.2020 primaries, unspecified transfer function, and the BT.2020 matrix. The value two is not a standard Apple Log enumeration. `logforge.transfer=Apple Log` is a project statement useful for auditing, but an editor cannot be assumed to recognize it. The logs payload `com.apple.rec2020.apple-log` is the identifier validated by the repository through reference files and controlled Resolve imports.

Substituting a generic log enumeration would declare a different curve, while retaining an HLG tag would conflict more directly with the transformed pixels. The current approach leaves the unavailable standard enumeration honestly unspecified and supplies more specific information through a separately validated extension. These layers are not redundant tags: they serve different interpretation layers.

15.2 The scope of historical identification experiments

`docs/APPLE_LOG_IDENTIFICATION.md` records native reference provenance, SHA-256, negative baselines, candidate fields, and actual Resolve import outcomes. The identification writer's validation report retains that experiment's software version and date and explicitly sets `this_file_imported_in_editor` to false. It indicates conformity of the current output structure to an established pattern, not that every transcode opens an editor for a fresh experiment.

The paper preserves this distinction. The controlled version 1.1.0 result in Resolve Studio 20.3.2.9 on Windows supports compatibility evidence for that configuration. Reusing the writer in version 1.2.1 shows that the mechanism continues, but does not automatically certify newer editors or other products. Apple Log and Apple Log 2 / Apple Wide Gamut are also different targets; similar names do not extend the scope.

15.3 Structure of the thirty-five-byte extension

`WriteToEncodedPartial` constructs a logs atom of thirty-five bytes: an ordinary eight-byte atom header followed by a twenty-seven-byte UTF-8 identifier. It contains neither a FullBox version-and-flags header nor a terminating NUL. This structure comes from actual references and isolation experiments, not an inference from API names. Validate requires exactly one HQ sample entry, one logs atom, and one `colr` atom, and checks that both extensions are direct children of that sample entry.

$$L_{\text{logs}} = 8 + 27 = 35 \text{ bytes} \quad (57)$$

If a complete correct identifier already exists, the writer returns immediately, giving idempotence. Conflicting or duplicate logs atoms are rejected instead of accumulating more. Conflicting sample-entry extensions such as `gama`, `mdcv`, and `clli` also fail validation. This prevents inconsistent color declarations from yielding different interpretations under different software precedence rules.

15.4 Why moov must be at the end

Inserting an atom increases file length. If `moov` precedes media data, shifting later bytes can change `mdat` offsets and require corresponding sample-offset-table updates, substantially increasing risk. The current writer accepts only one `moov` at the end of the file and rejects fragmented `moov`. It then needs to shift only the tail after the insertion point inside `moov`, leaving earlier media payload and packet offsets in place.

$$O_{\text{mdat}}^{\text{after}} = O_{\text{mdat}}^{\text{before}} \quad (58)$$

The ancestor chain from moov to apch is checked for actual containment at every level rather than assumed to have a fixed depth. Each ancestor's size grows by thirty-five. Overflow of an ordinary 32-bit length is rejected; extended lengths are updated in their original representation. Under these structural premises, unchanged media offsets can be established. The writer cannot be described as a general-purpose MOV editor for arbitrary layouts.

15.5 Direction and memory cost of moving the tail

To move an overlapping file region toward higher offsets, copying must proceed backward from the end. Otherwise, an early write can overwrite unread bytes. The code uses a 1 MiB buffer, moving backward from the file end toward the insertion point and writing each block at its old position plus thirty-five. It then writes the new atom, patches ancestor sizes, and flushes the file.

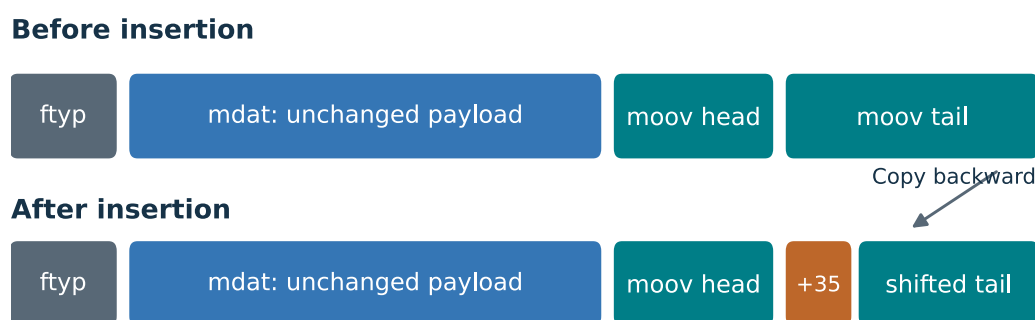


Figure 13. Insertion of logs into a trailing moov. The narrow colored block adds 35 bytes. Reading proceeds from right to left while writing to higher offsets, avoiding overlap corruption. The location of mdat is unchanged.

While a handle denying writes and deletion is held, other ordinary processes cannot replace the temporary file being analyzed and modified. Cancellation can occur before each move block; a failed temporary file is cleaned by the task and never published. The algorithm requires neither allocation of the entire moov nor re-encoding of video or audio. Failure midway can nevertheless leave an incomplete structure, so the transaction boundary must remain outside final publication.

15.6 Preserving provenance rather than impersonating a camera

The metadata allowlist may retain genuine make, model, creation_time, timecode, and language fields, but does not invent an Apple encoder or camera identity. The video-encoder identifier explicitly names LogForge / FFmpeg prores_ks. Apple Log identification describes numerical interpretation, not a claim that an Apple capture device generated the file natively.

This distinction also matters academically. Merely falsifying a camera tag until an editor displays Apple Log would not establish correct transfer curve, range, or encoding. The project separates color computation, identification fields, and provenance fields so those propositions can be tested independently. This paper likewise does not substitute an editor's displayed name for comprehensive correctness evidence.

Principal source files: [src/AppleLogIdentification.cpp](#); [docs/APPLE_LOG_IDENTIFICATION.md](#); [tests/identification.cpp](#).

Chapter 16. Metadata Allowlists, Creation Times, and Fidelity Semantics

16.1 From wholesale copying to an explicit plan

Copying all source metadata unchanged is simple, but may retain HLG, HDR, Dolby, PQ, or custom gamma information that is no longer valid. AppleLogMetadataWriter first disables automatic metadata copying at the relevant scopes, then restores a limited allowlist through CopyPlan. Every retained or removed item records its scope, key, value, and reason. Metadata fidelity is thus defined as conditional preservation rather than complete byte copying.

The allowlist includes creation time, genuine camera manufacturer and model, timecode, and language. Conflict detection lowercases key names and checks color-related word fragments. Even apparently harmless fields may be removed if not allowlisted. This conservative policy sacrifices preservation of unknown fields in return for a more auditable output interpretation. It does not mean every unknown field is dangerous; it means the project has not established a reliable preservation contract for it.

16.2 Scope promotion and conflict handling

Ordinary MOV video-stream tags may not serialize arbitrary camera keys. When safe and consistent with any same-named movie-level value, the code therefore promotes certain video-level camera fields to format scope and writes them through mdta. Creation time, timecode, and language follow dedicated rules. The copying policy determines where a value is written as well as which value is retained.

Output validation looks for each value at the plan's write_scope instead of mechanically searching only its original scope. Missing or semantically unequal values fail the task. One explicit exception treats default language und as equivalent when absent from the output. Exceptions need semantic justification and independent tests; otherwise an allowlist gradually becomes an unexplained collection of permissive matches.

16.3 Semantic equality of time strings

The same instant may be written as UTC with Z, with a colon-separated timezone offset, or with the colon-free offset common in QuickTime. For creation-time-related keys, SameMetadataValue attempts conversion to a common microsecond instant, accepting different strings that denote the same time. Dates, hours, minutes, seconds, timezone, and fractional parts are strictly validated. An uninterpretable timezone-free string is compared only as literal text.

$$t_{\text{UTC}} = t_{\text{calendar}} - \Delta t_{\text{zone}} \quad (59)$$

Nonzero fractional digits beyond microseconds are not silently truncated, preventing unequal times from being declared equal. Semantic normalization therefore has a boundary: it may remove representational differences, but must not manufacture equality through arbitrary rounding. Time preservation here means that designated fields denote the same instant within parseable precision, not that the strings must remain unchanged.

16.4 Header times differ from tag times

A movie-level creation_time tag is stored separately from structured creation seconds in mvhd, tkhd, and mdhd. Some muxing paths assign the global creation time to every track, losing independent source header times. PreserveMovCreationTimes uses source atom analysis to match target headers by movie and by handler type and ordinal, modifying only fixed-width fields.

This patch does not enlarge atoms, move mdat, or adjust sample tables. If a target header accommodates only 32-bit seconds and a source value exceeds that range, the function fails explicitly. The file is reparsed after writing and restored values are checked field by field. An identity display matrix is also written when rotation has been

baked; otherwise the recorded source matrix is restored. Preservation of creation information thus extends beyond a single metadata tag to container structure.

16.5 Timecode and chapters

Source timecode may reside in a video tag, a tmcd track, or a movie tag. MediaInfo collects it in priority order. The output timecode track is generated through an explicit timecode argument, rather than blindly copying an old track that may override new parameters. Timecode aids post-production alignment, but does not replace actual playback PTS. A file with a correct timecode string can still have audiovisual displacement.

Chapters are mapped from the source input, and titles are restored individually after automatic metadata copying has been disabled. Output validation compares chapter count, start and end times, and titles, using a one-millisecond time tolerance. Chapters, timecode, and audio start belong to different semantic layers. Separate checks are more reliable than comparing container duration alone. Similar total duration is not treated as sufficient evidence for fidelity of every temporal field.

16.6 Removal records are part of the result

For post-production, knowing that a field was deliberately removed is often more useful than silent loss. A report can explain that old HLG or HDR declarations were removed as conflicts, or that an auxiliary mebx track was omitted because it lies outside the output target. Users can then decide whether the original file must be retained as a metadata archive. A video transcoder and a general media-asset archiver need different preservation policies.

The chapter therefore establishes preservation under defined allowlist semantics and removal of conflicting declarations, not complete copying of all capture information. Workflows depending on gyroscope data, lens correction, depth, or vendor-specific tracks require separate verification. The repository's present scope does not justify presenting those capabilities as implemented.

Principal source files: [src/MetadataPolicy.cpp](#); [src/MediaSafety.cpp](#); [src/Media.cpp](#).

Chapter 17. Tool Discovery, Execution Trust, and Numerical Qualification

17.1 Discovering a path is not authorizing execution

Desktop applications often search PATH or disks for ffmpeg.exe. A same-named file may have unknown provenance, be a broken link, or have been replaced. LogForge separates path discovery from execution authorization. Discovery records candidate paths, pair status, and origin. Only an explicitly approved path-and-hash pair, or managed tools downloaded with a fixed hash, may enter capability checks. Requesting a version also executes a program, so trust checking must precede the version command.

This logic lies at the boundary between FFmpegManager and ToolTrust. HasApproval first queries records to avoid executing unknown programs during scanning. Acquire resolves canonical paths again, locks both executables, computes hashes, and compares them with approval records. Both ffprobe and ffmpeg are trust objects. Approving one cannot imply trust in any same-named probe found beside it.

17.2 Four-part identity and the lease

The executable pair's identity can be modeled as two canonical paths and two SHA-256 hashes. Path comparison alone misses replacement at the same location. Hash comparison without paired paths cannot easily express the installation location actually approved by the user. ToolLease holds two handles denying writing and deletion throughout execution, narrowing the window for replacement between checking and use.

$$\mathcal{I} = (p_f, p_p, H(f), H(p)) \quad (60)$$

This constrains execution identity under particular operating-system sharing semantics. It is not a proof that the machine withstands administrator-level attacks. A hash establishes equality of bytes with the approved object, not trustworthiness of its author or absence of vulnerabilities. The argument is only that the executed tools are bound to the recorded identities; the approval mechanism is not presented as a malware detector.

17.3 A finite budget for Quick discovery

Quick discovery shares a three-thousand-millisecond filesystem budget. Managed and saved paths have priority. If an approved, numerically verified pair is found, later sources are not traversed. Otherwise, the remaining budget covers nearby application directories, PATH, registry App Paths, actual package-manager installation directories, bounded-depth common folders, and an existing Windows Search index. Hashing and numerical qualification are timed independently and do not consume this filesystem budget. Three seconds is therefore not an absolute bound on the entire startup process.

DiscoveryHelper runs through a hidden entry point in the same executable and is supervised by its parent. This addresses disk or index queries that can block. An ordinary background thread cannot guarantee immediate cancellation of a stuck system call, whereas a separately supervised process provides a stronger termination boundary. Deep discovery enumerates accessible local disks only after explicit user selection and skips reparse points, offline areas, and protected regions.

17.4 Reporting search outcomes honestly

Discovery records actual directory counts, skipped counts, candidate origins, termination reasons, and diagnostics. A timeout or unavailable index is not reported as proof that the system has no FFmpeg. Candidates are capped at 256, and common-directory traversal has depth and entry limits. The number found describes observations within the current search scope, not a complete disk-wide asset inventory.

Path normalization also accounts for case, long paths, and mounted volumes. Windows known folders locate installation areas; the test-oriented LOGFORGE_DATA_DIR is not mistaken for every real user directory. Tests

inject roots and fake tools to check whether searching exceeds its scope or accidentally executes unknown programs. These critical branches can be verified without scanning all user disks.

17.5 Capability checks and numerical checks

Once execution is permitted, the program checks the ProRes decoder, `prores_ks`, `gbrpf32le`, `yuv422p10le`, and filters including `zscale`, `format`, and `setparams`, then runs small codec smoke tests. Even the presence of every name does not establish correct matrix, range, and sampling phase. Independent numerical signal tests are therefore required before the tools receive numerically Verified status.

Execution admission is a sequence of distinct gates

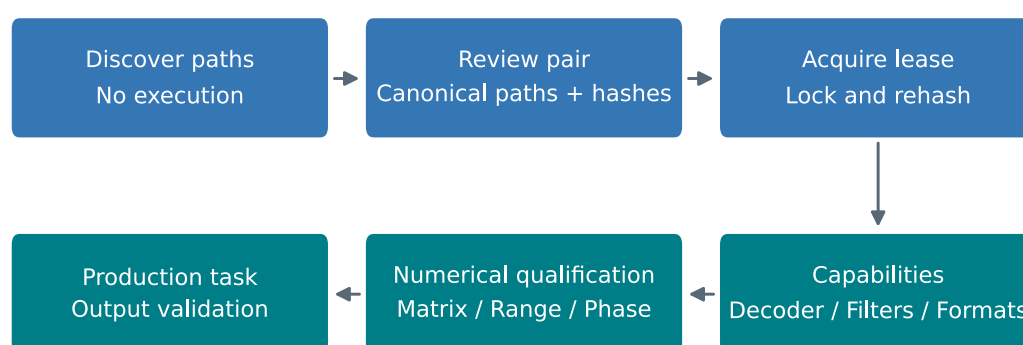


Figure 14. Admission of external tools. Each gate addresses a different question: bound identity, available functionality, numerical behavior, and validation of the actual task. Passing an earlier gate does not imply passing a later one.

TranscodeJob rechecks both the qualification flag and lease hashes, preventing a caller from bypassing discovery and submitting a merely compatible but numerically unverified tool. UI and CLI display state are therefore not the only defenses. The core itself verifies execution premises, reducing the risk of omitted checks when future entry points are added.

17.6 Fixed downloads and supply-chain boundaries

GyanReleaseProvider fixes the version, HTTPS URL, archive root, and SHA-256. The installer limits download size, checks integrity, retries at most three times, and validates in a staging directory before moving to managed storage. Archive entries must stay under the expected root; absolute paths, parent traversal, and drive identifiers are prohibited. A damaged old installation is retained as an ownership-marked backup rather than overwritten unconditionally.

A fixed hash avoids irreproducibility from a changing latest release, but makes upgrades explicit changes requiring repeated qualification. HTTPS protects transport, a fixed hash binds content, and runtime numerical checks constrain behavior. None substitutes for the others. This paper performed no new online installer download. The discussion relies on source and existing tests and does not invent a supply-chain audit outcome.

Principal source files: `src/ToolTrust.cpp`; `src/FFmpeg.cpp`; `src/Discovery.cpp`; `src/DiscoveryHelper.cpp`.

Chapter 18. Process Supervision, Pipe Deadlocks, and Cancellation Correctness

18.1 The supervised launch sequence

Platform.cpp creates a child suspended, assigns it to a Windows Job Object with kill-on-close, and then resumes its main thread. Starting a process before assigning supervision creates a brief escape window. Suspending first makes the process part of the supervised domain before it begins work. If the main application exits abnormally, closing the Job Object handle triggers cleanup of its member processes.

STARTUPINFOEX explicitly limits inherited handles to the three needed for standard input, output, and error. If every inheritable handle were passed through, a decoder might retain one end of an encoder pipe. Even after the real writer exits, the reader would then never observe EOF. Such hidden handle leaks often appear as a permanent wait after completion and cannot be meaningfully concealed by adding an arbitrary timeout.

18.2 Standard error must also be drained continuously

Pipes have finite capacity. If a parent reads only stdout while its child writes substantial stderr, a full stderr pipe blocks the child, which stops producing stdout while the parent waits for it. The transcoder uses separate threads to drain decoder errors, encoder errors, and encoder progress. RunProcess also reads standard output and error concurrently.

The wait graph contains a parent waiting for child output and a child waiting for the parent to free error-pipe capacity. Breaking that cycle requires an independent consumer, not just a slightly larger buffer. The current implementation also bounds line length and retained output size so an abnormal stream without newlines cannot exhaust memory. With a callback, data can continue to be processed as a stream rather than retained forever in a string.

18.3 Parent exit does not imply pipe closure

A tool can launch descendants that inherit its pipe handles. The original tool may exit while descendants keep the pipes open. RunProcess therefore supervises both the original process's running state and completion of the stdout/stderr reader threads. Cancellation and deadlines remain active after the original process exits, until the pipes truly finish.

tests/process_fixture.cpp constructs descendants holding pipes, long-line output, and different exit codes. reliability_process checks exception propagation, deadlines, and descendant termination. These fixtures are more targeted than a cancellation test using only a normal ffmpeg invocation because they directly exercise resource-lifetime relationships that can cause hangs.

18.4 The order of cancellation and cleanup

User cancellation sets an atomic flag. A cancellation observer checks at short intervals and terminates both Job Objects. Blocked reads and writes can then exit and return exception or cancellation state to the main task. All threads are subsequently joined, and temporary output owned by the task is removed. Window closure follows the same path, avoiding an invisible encoder continuing after the interface disappears.

Cleanup must first give external blocking operations an opportunity to finish, then wait for threads. Joining a thread that reads a pipe which never closes would prevent a later termination statement from ever running. Several RAII guards encode this principle and cover exceptions after only some threads have been created. Destructor safety in concurrent software belongs among the design premises, not as an error-handling afterthought.

18.5 Exit codes and exception precedence

The task checks user cancellation, progress-thread and error-reader exceptions, color-processing exceptions, decoder exit code, encoder exit code, and frame count. A zero process exit code cannot erase a reader exception; a correct frame count cannot erase a numerical failure. Preserving the first exception helps identify the root cause, while logs retain exit status and external diagnostics for analysis of cascading failures.

Progress data is itself bounded text. frame, fps, out_time_us, and speed are parsed separately. Failure to parse one value should not produce a falsely precise time estimate. Some stages, including identification writing and final validation, permit indeterminate progress. The interface should display their names rather than compress all work into a seemingly continuous, always trustworthy percentage.

18.6 What supervision can and cannot establish

Assuming the operating system implements documented Job Object, handle-sharing, and pipe semantics, one can argue that child processes share a cleanup domain, read/write threads have cancellation-release paths, and unvalidated temporary media is not deliberately published. These mechanisms alone cannot establish immediate reclamation of every system resource after an arbitrary driver hang, kernel fault, or power failure. The failure model is limited to what the application can control, with operating-system services explicitly treated as assumptions.

This boundary makes the conclusion testable rather than weakening it. A claim that software can never hang, without assumptions about system calls, drivers, and hardware, is harder to verify than specific tested exit conditions. LogForge's supervision is best explained through resource ownership and state transitions, not unqualified reliability adjectives.

Principal source files: [src/Platform.cpp](#); [src/Transcode.cpp](#); [tests/process_fixture.cpp](#); [tests/reliability.cpp](#).

Chapter 19. File Transactions, Disk Planning, and Crash Recovery

19.1 Separating final and temporary paths

TranscodeJob requires a new MOV destination and creates a unique partial in the same directory using the process ID and a time counter. Conversion, metadata repair, and validation all act on the partial. The destination name receives content only after every condition passes. CREATE_NEW reserves the temporary path to avoid occupying an existing file accidentally. Final publication uses MoveFileExW without REPLACE_EXISTING.

An initial check that the destination does not exist is insufficient, because a user or another process may create the name during a long conversion. The final no-overwrite move asks the operating system to check the conflict again, closing the familiar race between an absent target at check time and an overwritten file at write time. Publication failure remains task failure. Completed encoding does not authorize overwriting the destination or silently choosing another name.

19.2 The publication invariant

A safety condition can describe the relationship between this task and final-file publication: the task attempts publication only after validation and successful report saving, and only when not cancelled. This does not imply that every same-named file in the directory was produced by this task, because other processes may create one. The argument concerns the task's own writes, not the global state of a shared directory.

$$\text{PublishAttempt} \Rightarrow \text{Validated} \wedge \text{ReportSaved} \wedge \neg \text{Cancelled}$$

(61)

Validation precedes final-file publication

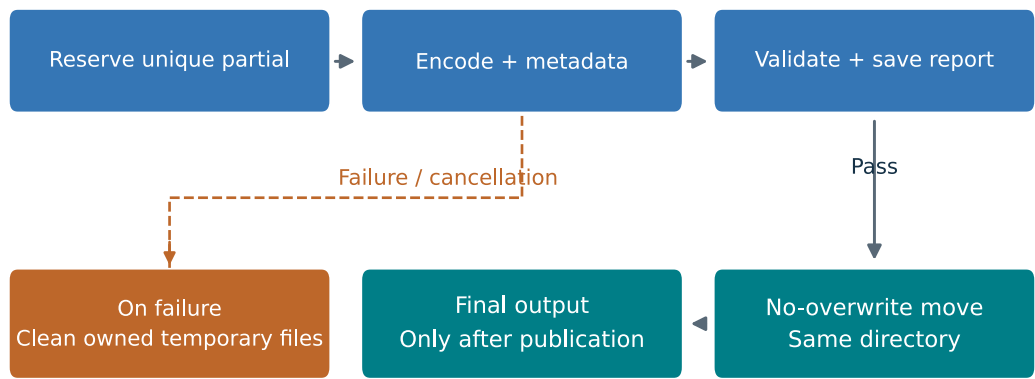


Figure 15. Task states and the publication boundary. Encoding is followed by metadata repair, output validation, and report persistence. Failure returns to cleanup. Only the final no-overwrite move turns temporary media into the delivered file.

A same-directory move favors same-volume semantics and avoids copying the entire file. WRITE_THROUGH expresses an intent to persist the write, but is not treated here as a database-grade transaction guarantee across every filesystem, caching device, and sudden-power-loss scenario. The selected API, filesystem behavior, and hardware durability are different layers.

19.3 Disk requirements are estimated heuristically

The space preflight starts from approximately 220 Mb/s for 1080p ProRes HQ at about 29.97 fps, scales with pixels, frame rate, and duration, multiplies by a 1.75 margin, and adds estimated audio and fixed headroom. A second copy is considered when rotation remuxing is needed. Scene complexity and encoder behavior affect actual bitrate, so the estimate is a planning value for identifying obvious insufficiency early.

$$B_v \approx \frac{220 \times 10^6}{8} D \frac{WH}{1920 \cdot 1080} \frac{f}{30000/1001} 1.75 \quad (62)$$

Even after preflight passes, another process may consume space, a device may disconnect, or actual bitrate may be higher. Runtime write errors must therefore remain fatal. `tests/v120.cpp` uses `JobIOHooks` to inject low available space and report-write failure, verifying that failed tasks do not publish final media. Keeping these hooks in a test-injection interface instead of ordinary user environment variables reduces accidental alteration of production behavior.

19.4 Ownership of temporary files

A partial left by a crash cannot be deleted merely because its filename matches a pattern. `StorageSafety` journals the process ID, process creation time, absolute path, volume serial number, file index, and file creation time. Recovery first establishes that the original owner is dead or its process ID has been reused, then checks that the path still refers to the recorded file identity.

$$\text{MayDelete} = \text{OwnedJournal} \wedge \text{DeadOwner} \wedge \text{SameFileIdentity} \quad (63)$$

Process IDs are reused, so `process_start` is necessary. A path may later refer to a replacement file, so file identity is necessary as well. Failure to open a process because access is denied is not taken as proof of death, and identity mismatch prevents deletion. This conservative policy prioritizes avoiding deletion of user media, at the cost of leaving some uncertain remnants for manual handling.

19.5 Concurrent locking of state storage

Updating settings or trust consists of read, modify, and write. Atomic replacement of the final file alone does not prevent two processes from losing one another's updates. `StateLock` holds an exclusive byte-range lock in a data-directory lock file and combines it with an in-process mutex to serialize the read-modify-write sequence. A temporary file in the same directory then replaces the official file. Windows releases file locks when a process exits, supporting recovery after crashes.

Saving preferences updates only the fields owned by that operation and retains separately saved FFmpeg paths. Saving paths likewise preserves other preferences. Field-level merging reduces interference among features. If two windows edit the same preference, however, the last saved value still wins; the application is not a collaborative live editor. The concurrency granularity should therefore be stated instead of ending the analysis with the generic phrase thread-safe settings.

19.6 The scope of retention and cleanup

Maintenance removes only recognized log names and installation directories after checking age, ownership, and reparse points. It does not recursively search user media folders and delete MOV files. Logs generally have a thirty-day retention threshold, while marked old installation-staging directories use a separate threshold. An old partial without a journal does not acquire deletion authorization merely through a similar name.

This design ties convenience to provable ownership. In large-media workflows, cleanup itself may be more dangerous than a failed conversion. Recovery and retention should therefore not be treated as incidental code that simply deletes temporary files. They require path, identity, and concurrency assumptions as careful as those of publication.

Principal source files: `src/StorageSafety.cpp`; `src/Settings.cpp`; `src/Transcode.cpp`; `tests/v120.cpp`.

Chapter 20. GUI, CLI, Queues, and Inspectable Interaction

20.1 A single owner of background work

MainWindow uses one background worker for discovery, download, validation, transcoding, or queue operations. Busy state disables controls that would conflict with the active task. The UI thread constructs immutable task options before handing them to the worker. Changing settings during processing therefore cannot cause the first half of a video to use one parameter set and the second half another.

Workers send events through custom WM_APP messages containing type, progress, media, tools, errors, or reports. PostMessage transfers ownership of heap objects, and the UI consumes them through unique_ptr. This centralizes cross-thread state changes in message handling instead of allowing background threads to manipulate controls arbitrarily. Shutdown must also handle unconsumed events and worker completion to avoid lifetime mismatches.

20.2 Layout driven by measured text

A single layout owner computes control rectangles and wrapping results. Text measurement and drawing use consistent line breaking, avoiding duplicate status rendering by the background and controls. Changes to DPI, window size, language, theme, or state trigger layout updates. A scrolling viewport keeps lower controls reachable in small windows and at high scaling, adjusting scroll position when keyboard focus moves outside the visible region.

Ui.cpp manages fonts, brushes, colors, native-control appearance, and WIC screenshots. System high-contrast mode takes priority over the project theme. DWM title-bar attributes are requested when available, without depending on private dark-mode APIs. Usability depends on readable diagnostics, reachable buttons, visible focus, and a single source of status, not simply on an attractive palette.

20.3 Settings drafts and cancellation semantics

SettingsWindow stores edits in draft. Only Save calls SettingsStore and sets accepted to true. Cancel destroys the dialog without writing the draft back. The creative switch modifies the draft and refreshes control availability; language and theme choices also remain provisional. This model of withholding changes from active configuration until commit resembles file publication conceptually, but applies to user preferences.

Tests first open and modify a draft, cancel, and verify that preferences did not leak. They then save another parameter set and check persistence and application. Binding controls directly to mutable global settings could make Cancel merely close a window without reversing changes already made. The draft object is therefore required by observable behavior, not just by a preferred code organization.

20.4 Stable message keys and translation

Messages.inc defines message identifiers and bilingual text through a macro table. Localization.cpp checks unique keys, complete entries, and matching placeholder sets. Workers pass TextId and arguments, while the UI translates using the current language. JSON retains stable error codes, and logs and raw tool diagnostics retain English. Error codes serve automation; translations serve readers.

Placeholder substitution is a single pass. Inserted paths and error strings are not subsequently interpreted as templates. A filename containing text resembling a brace-delimited zero must be displayed literally, not trigger another replacement. This is an input boundary: user-controlled strings remain data and must not become control templates after formatting.

20.5 Serial queues and per-item outcomes

RunQueue derives an independent output name for each input, probes each item, and calls the same TranscodeJob. A failed item records an error and failure report before processing continues with the next item. User cancellation stops further work. Results include successful, failed, and remaining counts and cancellation state. The queue does not run several media tasks simultaneously, so it does not multiply the three-stage resource budget by the file count.

The default output name uses the source stem plus an AppleLog suffix and does not overwrite automatically. Same-named files from different origins may collide at the destination. The corresponding item then fails rather than overwriting an earlier result. Queue explanations should state this clearly: batching improves operational efficiency but does not remove naming collisions or disk-planning requirements.

20.6 The automation value of the CLI

Cli.cpp exposes detect, probe, convert, batch, qualify-cuda, and analyze. Argument parsing validates command shape and relationships among creative options before discovering or executing tools. Specifying creative parameters without enabling tone is rejected, preventing users from assuming that inactive adjustments took effect. Exposure and creative values must parse completely, be finite, and lie within range; a number with trailing garbage is not partially accepted.

The CLI and GUI share mathematical, temporal, and publication rules through the core, but their exposure controls differ. The GUI offers discrete values from -4 to +4 stops; the CLI accepts -8 to +8. Reproducing experiments requires naming the entry point and all parameters rather than merely saying default settings. Defaults may refer to different configurations at different times, under saved state, or through different interfaces.

Principal source files: [src/MainWindow.cpp](#); [src/SettingsWindow.cpp](#); [src/Ui.cpp](#); [src/Localization.cpp](#); [src/Cli.cpp](#); [src/Queue.cpp](#).

Chapter 21. Test Structure, the Present Reproduction, and Failure Analysis

21.1 Tests are not a single score

Repository tests cover mathematics, media parsing, tool trust, process supervision, cadence, MOV identification, parallel consistency, settings, resources, and actual codecs. Combining them into a pass rate gives a quick overview but does not explain how much evidence supports each research question. A pure mathematical test does not establish audio synchronization, and a GUI screenshot does not establish conformity of pixels to formulas. Tests are grouped by contract here rather than presented merely as green pass marks.

`tests/tests.cpp` covers color reference points, dense round trips, parameters, and finiteness. `hardening.cpp` checks time, trust, MOV, and parallel paths. `reliability.cpp`, `process_fixture`, and `fake_ffmpeg` cover abnormal processes and non-execution of unknown tools. `application.cpp` covers preferences, discovery, and resources; `identification.cpp` directly checks identification-writing structure; `v120.cpp` adds storage and backend fault injection and covers queues and metadata.

21.2 The role of real-media tests

`integration.py` generates controlled media and checks actual output. `signal_integrity.py` independently validates color using integer patches. `sustained.py` processes 240 consecutive 4K120 frames and checks memory and sampled luma. `compatibility_v121.py` uses generated structures to cover portrait pixels, identity matrices, and audio offsets. These test end-to-end behavior of the production core rather than merely calling a simplified substitute.

`gui_smoke.py` and `layout_snapshots.py` require an interactive desktop and real windows to check language, theme, DPI, interaction paths, and screenshots. `resolve_identification.py` requires an actual Resolve scripting connection and suitable references, making it an editor-interoperability experiment. Historical outcomes from these latter categories are not presented as rerun results here, and simulated windows or fabricated editor responses were not used as substitutes.

21.3 The build and complete CTest run

On September 29, 2026, a Release build was executed in the current Windows workspace and every configured target built successfully. The twenty-four CTest entries in the current build configuration were then run, including actual CUDA qualification, media conversion, portrait compatibility, signal, publication, and sustained 4K120 tests. Twenty-three passed and one failed, with a total test wall time of approximately 195.71 seconds. The CUDA artifact-consistency script was also executed and passed checks of source and PTX hashes, target architecture, and precise compilation options.

Test logs and JUnit XML accompany the paper. The XML's top-level time field is an integer number of seconds as emitted by the tool; individual timings retain decimals. The total in the narrative comes from CTest's terminal summary. Source or tests were not modified and rerun to obtain an all-green outcome: the fixed revision remains unchanged. Failure is part of the research object and must not be hidden in an effort to prove the tool correct.

Table 3. Complete CTest results from the reported reproduction

Test name	Outcome	Time (s)
robustness_v120	Passed	0.940
cuda_qualification	Passed	0.133
apple_log_identification	Passed	0.557
reliability_process	Passed	1.762
reliability_discovery	Passed	4.128
reliability_cli	Passed	1.597
hardening_cadence	Passed	0.085
hardening_security	Passed	0.540
hardening_mov	Passed	0.051
hardening_parallel	Passed	2.486
color_math	Passed	0.192
media_metadata_validation	Passed	0.015
ffmpeg_platform	Passed	1.007
application_preferences	Passed	0.265
ffmpeg_disk_discovery	Passed	0.084
application_resources	Passed	0.045
portrait_storage_queue_v120	Passed	36.855
compatibility_iphone_v121	Passed	6.448
compatibility_blackmagic_v121	Passed	5.188
media_pipeline	Failed	29.726
color_signal	Passed	14.229
ffmpeg_discovery_capabilities	Passed	5.351
publication_no_overwrite	Passed	5.461
sustained_4k120	Passed	78.546

21.4 Why media_pipeline failed

The failing assertion is at line 154 of tests/integration.py. The test copies a valid source into a MOV with only color primaries changed to bt709, retaining ProRes and HLG conditions, and expects conversion to be rejected with no destination file. Version 1.2.1 UnsupportedReasons no longer checks primaries. The file is therefore accepted, processed under the explicit BT.2020 NCL interpretation, and passes production output validation. The test consequently reports Unsupported input accepted.

This is a contract conflict directly explained by the code. The older integration test expects stricter admission of color tags, while current release notes deliberately relax the format gate. Current unit tests already assert that missing optional tags do not block input, but this older end-to-end expectation was not synchronized. The immediate maintenance question is whether explicitly conflicting primary tags and missing tags should receive different policies. Implementation, tests, and documentation should then agree. This paper analyzes the issue without unilaterally changing repository policy.

21.5 Why the failure is not simply harmless

From a test-maintenance perspective, an outdated expectation caused the failure. From a signal-interpretation perspective, it also exposes the practical cost of permissive admission. An HLG file explicitly tagged BT.709 is processed as BT.2020. If the tag accurately describes its content, color may be wrong. Production output validation builds its reference from the same input interpretation and cannot independently prove that ignoring the source tag was justified. This is a shared modeling assumption, not a problem eliminated by output-format checks.

The twenty-three passes therefore do not establish that the tool has no problems, and the single failure does not establish that all conversions are unreliable. More precisely, mathematics, parallel execution, tool qualification, and several media paths have evidence from this environment. The complete integration suite retains an unresolved inconsistency where the input contract evolved, and the policy needs clearer user communication and distinct tests. Because `integration.py` stopped at the assertion, its later variants did not execute in this run and cannot be counted as covered.

21.6 Specific sustained-test results

The `sustained_4k120` reproduction used generated footage of two seconds, 240 frames, 3840 by 2160, and 120 fps. Conversion took 43.449496 seconds, approximately 5.523654 fps. Peak application private memory was 15142912 bytes, approximately 14.44 MiB. Frames 0, 119, and 239 were sampled, producing 1536 luma samples. Maximum post-encoding error was 0.86505094 codes, below the test's two-code threshold.

This conversion time differs from the 78.55 seconds reported by CTest for the case because the latter includes fixture generation, additional sampling, and script overhead. It also does not mean real-time 4K120 processing. Input acquisition rate is 120 frames per second; actual offline conversion throughput is approximately 5.5 frames per second. The two uses of fps describe different concepts and must be distinguished in figure titles and axes.

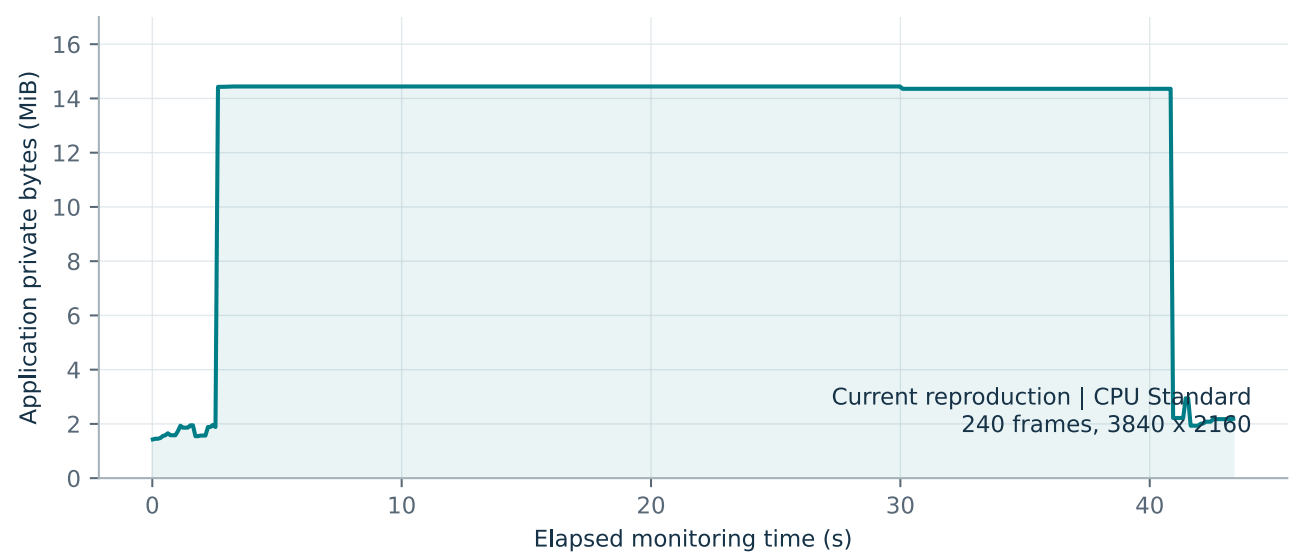


Figure 16. Application private-memory trace from the paper's 240-frame 4K120 reproduction in CPU Standard mode. Data comes from `sustained-report.json` and covers only the main application, excluding FFmpeg children and GPU device memory. This is an observation from one run.

Principal source files: `CMakeLists.txt`; `tests/integration.py`; `tests/sustained.py`; `docs/VALIDATION.md`.

Chapter 22. Performance Data, Complexity, and Reproducible Comparison

22.1 Workloads in the historical matrix

docs/benchmarks/1.2.0.json records twenty formal cases: 1080p24 and 4K at 24, 30, 60, and 120 fps, each combined with Standard/Creative and CPU/CUDA. Input is a one-second generated grayscale ramp. Each task performs tool checks and production output validation, with no audio or rotation. The dataset is suitable for comparing backend and workload costs on that machine, not for inferring throughput on long clips, complex color texture, disk arrays, or thermally stabilized hardware.

The hardware record identifies twenty logical CPUs, an RTX 3080 Ti Laptop GPU with sixteen GB of device memory, and a specified NVIDIA driver. The paper plots these as historical measurements and keeps their version and date explicit. A dataset named 1.2.0 cannot be relabeled as a new version 1.2.1 measurement. Even with an unchanged color kernel, thread allocation, operating-system state, and surrounding processing can affect timing.

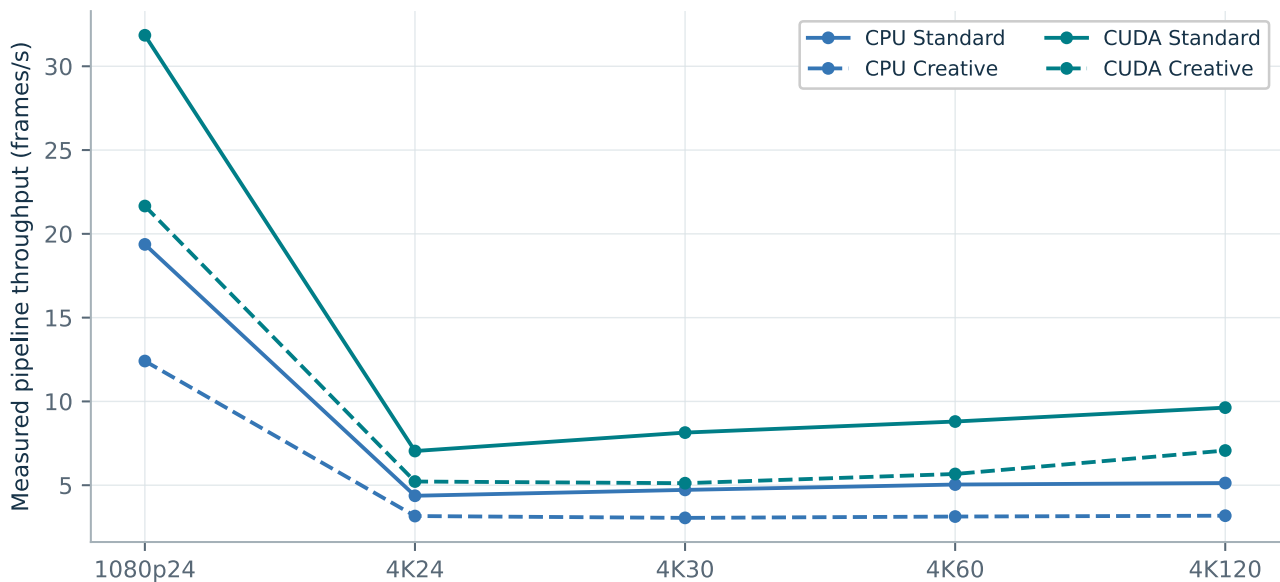


Figure 17. Historical throughput matrix from repository version 1.2.0. Solid and dashed lines distinguish Standard and Creative modes; colors distinguish CPU and CUDA. The vertical axis is measured pipeline throughput, not input capture rate. Each case is a single observation on one machine.

22.2 Calculating speedup from historical values

Historical 4K120 Standard pipeline times were 23.387 seconds on CPU and 12.465 seconds on CUDA, a ratio of approximately 1.876. Corresponding CLI totals were 27.336 and 16.405 seconds, giving end-to-end speedup of approximately 1.666, below pipeline speedup. Startup, qualification, metadata, and validation costs did not shrink proportionally. Any speedup report must identify its denominator and timing scope.

$$S_{\text{pipeline}} = \frac{23.387}{12.465} \approx 1.876, \quad S_{\text{CLI}} = \frac{27.336}{16.405} \approx 1.666 \quad (64)$$

For the same input, historical Creative pipeline times were 37.734 and 16.985 seconds, a ratio of approximately 2.222. A larger ratio than Standard mode does not mean Creative is necessarily faster. The CPU Creative path is heavier, so CUDA accelerates more computation. Absolute duration and relative speedup are different metrics and should preferably be reported together to avoid misleading selection of the highest ratio.

22.3 Amdahl's law and bottleneck migration

If the accelerable fraction of total work is p and local speedup is s , ignoring additional costs gives the following overall limit. In LogForge, ProRes codecs, matrix filters, container operations, and validation remain on CPU or disk. Even an infinitely fast color kernel leaves these costs as a limit. GPU transfer overhead lowers the practical limit further.

$$S_{\text{total}} \leq \frac{1}{(1-p) + p/s} \quad (65)$$

Pipeline overlap also changes the interpretation of p . Stages are not all added serially, so shortening a stage may remove only part of the wall time. Reported read and write times include backpressure; decoder/encoder CPU seconds are accumulated CPU time; GPU event time is device time. Adding them directly does not produce total elapsed time. Their definitions are retained instead of constructing an invalid additive pie chart.

22.4 Memory comparison requires a stated scope

Historical 4K120 CPU Standard application peak memory was approximately 14.4 MiB, but the process tree reached approximately 2110.7 MiB. CUDA Standard used about 297.9 MiB in the application and 2768.5 MiB in the process tree. CUDA Creative used about 565.6 MiB and 2866.2 MiB respectively. Low memory use by a small native GUI does not imply that complete transcoding requires only a few tens of MiB.

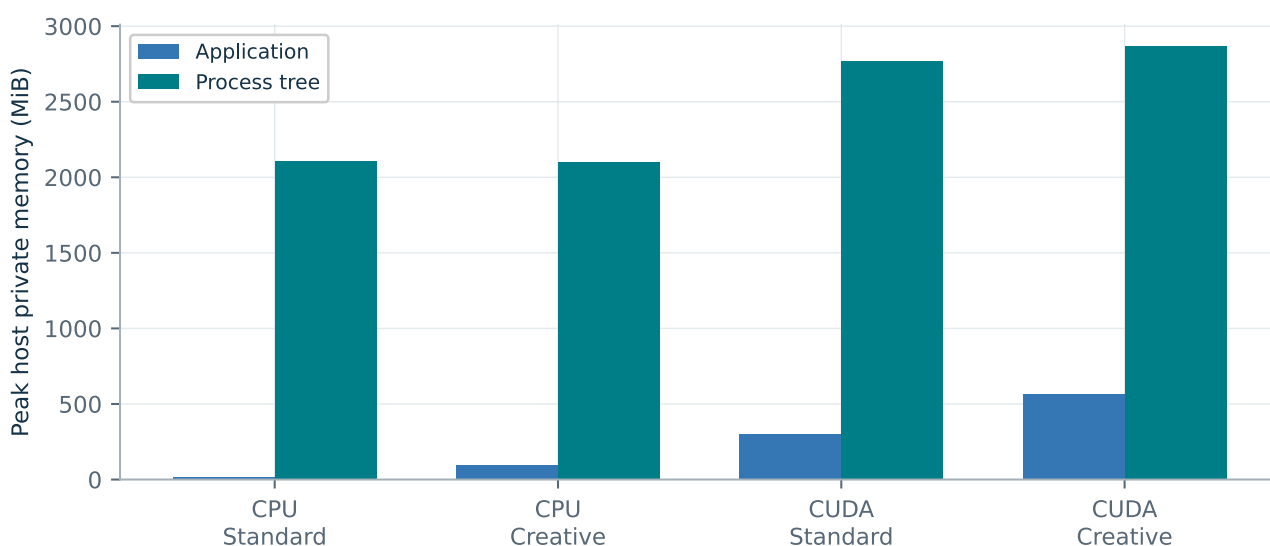


Figure 18. Historical version 1.2.0 peak application and process-tree memory for four 4K120 configurations. Both bar groups use the same units. GPU device buffers are reported separately; host private memory is not a measurement of video memory.

Theoretical buffer bounds and measured private bytes should be reconciled rather than substituted for one another. The standard bridge's twelve MiB explains the main application's principal pixel buffers, but thread stacks, DLLs, drivers, and allocators add overhead. Complexity expressions explain scaling with input size, while measurements expose constants in a particular environment. A reliable performance account needs both.

22.5 Improving a reproduction study

Cross-device conclusions require repeated runs with medians, quantiles, and dispersion, together with power mode, temperature, background load, disk, and driver records. Inputs should include smooth ramps, noise, strong texture, sharp color edges, and real camera clips. Generation, validation, and conversion times should be reported separately. Long tests should distinguish cold start, steady state, and thermal throttling instead of extrapolating a one-second clip linearly to an hour.

The paper invents no confidence intervals and does not interpret small differences between one new test and one historical test as a version-level speedup. The present reproduction took approximately 43.45 seconds, compared with about 45.20 seconds for the historical overlapping bridge. Environmental noise and measurement conditions can affect a difference of that size. Both may be reported, but they do not establish software optimization. This restraint makes the performance conclusions assessable by an independent reproducer.

Principal source files: [docs/BENCHMARK_1.2.0.md](#); [docs/benchmarks/1.2.0.json](#); [tests/benchmark.py](#).

Chapter 23. An Implementation Walkthrough from One Sample to the Final File

23.1 Establishing an explicit input example

Consider a two-second, 3840 by 2160, 24 fps ProRes HQ HLG MOV using BT.2020 NCL, video range, and left chroma siting, with a ninety-degree display orientation and 48 kHz two-channel PCM. Video starts at zero and audio at 500/48000 seconds. Exposure is zero, Creative processing is disabled, and CPU is forced. This is an instructional example, not an undisclosed camera original.

Probe first extracts the media summary and format admission passes. If the orientation matrix is a supported cardinal rotation, output geometry becomes 2160 by 3840. Source packet scanning should identify forty-eight frames and a fixed period. A partial is created only after space preflight and log-writability checks pass. No pixels have changed yet, and rejection at this stage produces no final output file.

23.2 Tracing a neutral sample

Choose an HLG neutral pixel near 18% gray. If H takes the reference value 0.3782588831, inversion gives a relative scene quantity of approximately 0.047693; multiplying by S returns 0.18; encoding then gives P approximately 0.4882724585. Equal channels imply zero output chroma, with unquantized luma code approximately 491.7267. Integer input can only approximate this H , so actual decoded output also includes quantization and compression differences.

$$H \xrightarrow{E} E(H) \xrightarrow{S} 0.18 \xrightarrow{f} 0.4882724585 \xrightarrow{64+876P} 491.7267 \quad (66)$$

The chain is a numerical explanation, not a requirement that each real pixel end at code 492. Chroma reconstruction depends on neighbors, ProRes introduces compression error, and error diffusion may place adjacent pixels at different integers. A uniform fixture, specified sample positions, and an explicit error metric are needed to turn the instructional chain into a valid test.

23.3 Tracing a memory block

The decoder emits the rotated G, B, and R planes in order. The first Standard block holds up to a little over one million float components. The reader marks it ready; color workers partition and transform it while updating statistics; the writer sends it to the encoder. Second and third blocks may occupy different stages concurrently, while byte order remains strict. Total sample count should ultimately be 2160 times 3840 times 3 times 48.

An incomplete float in the final block is rejected by the reading stage. A sample count that is not a whole-frame multiple is rejected at bridge completion. A frame count other than forty-eight is rejected by task-exit checks. These separately observe byte integrity, frame structure, and the media-time contract. Layered repetition is not redundant because each layer catches a different form of damage.

23.4 Tracing audio and the container

The encoder takes video from the pipe and copies audio from its second source input. The new video clock uses the validated 24 fps, while audio retains its 1/96-second relative start. The source movie timescale is passed to the muxer, creation headers are restored later, and timecode is reconstructed if present. Since portrait orientation has already been applied to pixels, the final display matrix should be identity.

After encoding, logs is written, nccl 9/2/9 is checked, the output is probed again, and output packets are scanned. PixelSanity compares scalar reference values with actual decoded results in selected blocks from three frames. Signal statistics report lower-floor clipping and above-nominal-white values. Only after those conditions and report saving succeed is the partial moved to the user-selected name.

23.5 Tracing a GPU failure branch

Change the same example to Auto. If initialization fails, CPU is chosen from the outset. If CUDA fails on a later block, the original host block still holds HLG input before commit. Auto sends it to CPU, and subsequent blocks continue on CPU. The report records `fallback_reason` and `cpu after cuda`; the whole clip must not remain labeled as a pure CUDA success. With forced CUDA, the same exception terminates the task instead of silently degrading.

Backend selection is therefore part of result provenance. Work may cross two implementations. With valid block commits and reference qualification, the color contract can still hold, but performance reporting must reflect mixed execution. Swallowing the exception and displaying only completion loses diagnostics and leaves users unable to explain different durations under apparently identical settings.

23.6 Tracing a final-path conflict

If another process creates the destination during conversion, the initial absence check is stale. The final `MoveFileExW` forbids replacement, so the task reports `OutputExists`, preserves the externally created file, and cleans its own temporary media. A passing validation report can therefore exist without a final video published by this task. External scripts must not interpret the report's `passed` field as a guarantee of publication.

This suggests a future interface refinement. The report is currently saved before publication, while publication-failure details reside mainly in the task error and log. An explicit publication state or a final transaction-result update would help automation distinguish content validated but publication failed from fully completed. This is an improvement proposal, not a claim that the current JSON already implements that field.

Principal source files: [src/Transcode.cpp](#); [src/FloatBridge.cpp](#); [src/color/CudaTransformer.cpp](#).

Chapter 24. Discussion, Implementation Limits, and Research Conclusions

24.1 The principal arguments supported by the analysis

The first argument concerns mathematical consistency. Inverse HLG, reference-white scaling, exposure, and Apple Log encoding are composed in an explicit order. Creative mode has a separate switch and luminance model. Piecewise derivatives, the gray-card fixed point, saturation's luminance preservation, and a positive derivative bound for the creative neutral curve follow directly from the formulas. Rounded joins, the negative extension, and the floor plateau supply necessary qualifications. Mathematical statements are mapped to source functions so readers can cross-check symbols and implementation.

The second argument concerns consistency of execution structure. The Standard transform is componentwise independent and permits bounded blocks. Creative processing depends on three channels and requires correct pairing. CPU tiles and the CUDA grid preserve this dependency structure. CUDA commits only after full synchronization and checking, preventing double transformation during Auto fallback. Ownership transfer in the three-slot state machine preserves order. These properties explain behavior more deeply than a single correct output example.

The third argument concerns delivery constraints. Timestamp validation supplies the premise for a timestamp-free pipe. Relative audio timing and media metadata have explicit checks. MOV writing restricts structure to preserve media offsets. Tool leases bind the executed bytes. Final publication depends on validation and a no-overwrite move. Each addresses a concrete failure mode, and together they establish engineering correctness of a media task rather than merely decorating a color formula.

24.2 Claims not established by the current evidence

The paper cannot establish recovery of original sensor dynamic range, complete equivalence to native-camera Apple Log, preservation of all fine texture, compatibility with every editor, or real-time high-frame-rate 4K processing. Such claims either conflict with irreversible information loss or require paired capture, perceptual evaluation, or cross-device experiments that were not performed. The repository likewise does not provide complete supporting evidence for them.

Nor has all C++ concurrency or every Win32 error branch been formally verified. The paper offers conditional arguments based on inspectable implementation structure, numerical derivations, and test evidence, not a whole-program theorem generated by a proof assistant. Formula-derived curves, single historical measurements, and the present single reproduction must each be understood according to their own evidence class.

24.3 The input contract most in need of clarification

Version 1.2.1 relaxes admission for a clear compatibility purpose, but missing information, explicitly conflicting information, and uninterpretable information should be distinguished. Missing camera model need not affect color computation. Defaulting missing chroma location to left may be pragmatic. Explicitly declared alternative primaries followed by forced BT.2020 interpretation creates a different risk. A single category of optional tags does not express these distinctions adequately.

Compatibility could be retained while recording defaults, alerting users to explicit conflicts, and giving different policies stable report fields. Tests should separately cover missing-but-allowed, conflicting-and-warned-or-rejected, and explicit user override. The failed integration test provides a concrete starting point: revise the contract, then synchronize code and tests, instead of deleting the assertion or restoring an old gate merely to obtain green results.

24.4 A stronger scientific validation program

If the research objective becomes camera-appearance equivalence, paired HLG and native Log captures should use the same scene, exposure, white balance, lens, and traceable settings. Grayscale targets, color charts, and high-dynamic-range scenes should be compared under unified decoding ranges and display transforms. Luminance, color difference, noise, and local-texture metrics should be reported, along with whether camera modes share an ISP path. Screenshots from two player windows cannot isolate those variables.

For editor interoperability, a matrix of software, version, color-management mode, and data level should include negative baselines, logs-only variants, other candidate fields, and real outputs. Both automatic identification and rendered values require checking. Performance requires repeated long tasks, varied texture, and multiple devices. Reliability requires broader fault injection, filesystem anomalies, and malformed-container coverage. Different research questions need different experiments; one successful conversion cannot answer them all.

24.5 Conclusion

LogForge demonstrates an understandable engineering approach: public color mathematics defines the transform, independent multimedia tools provide codecs, bounded memory and explicit ownership organize concurrency, and structured validation determines delivery. Its value lies not in inventing a new Log curve but in connecting existing standards with the timing, metadata, and failure behavior of a practical desktop workflow while leaving inspectable evidence.

In this fixed-version study, the Release build and twenty-three tests passed; one older rejection expectation conflicted with newer input admission; and sustained 4K120 numerical samples met the current threshold. These outcomes support operation of the core transform and several reliability mechanisms as designed in this environment, while retaining limits concerning input interpretation, evolving documentation, and finite sampling. A rigorous implementation paper must explain why success follows and where its conclusions cease to apply. On that basis, the paper develops a layered argument from formulas and code through tests to the final file.

Principal source files: [README.md](#); [docs/RELEASE_1.2.1.md](#); [docs/VALIDATION.md](#).

Appendix A. Symbols, Units, and a Review Guide

A.1 Unified notation

Table 4. Principal symbols and units

Symbol	Meaning	Unit or domain
H	HLG-encoded component	Normalized; nominally zero to one
E	Relative scene quantity from inverse HLG OETF	Relative quantity
S	Reference-white normalization scale	Approximately 3.7741181185
e	Exposure adjustment	EV; CLI range -8 to +8
R, G, B	Scene-linear components after exposure	Reference-reflectance coordinates
P	Apple Log-encoded component	Normalized
Y, Cb, Cr	Nonlinear matrix components	Normalized
q	Code value before storage or after decoding	10-bit code units
W, H (geometry)	Frame width and height	Pixels; distinguish H by context
tau, T	Time base and candidate packet period	Seconds per tick; ticks
N	Frame or sample count	Identified by subscript

A.2 An order for checking the formulas

A reviewer can begin with the constants in Color.h, checking Apple Log at zero, the join, middle gray, and the upper reference. Next check HLG's value and derivative at one-half, then the combined mapping of reference white and 18% gray. After those three stages, add YCbCr offsets and scales. This order separates transfer-function, exposure, and range errors instead of confronting the complete pipeline without a way to locate discrepancies.

For Creative mode, first disable saturation changes and use neutral grays to check monotonicity and the fixed point. Then introduce color patches to examine the weighted-luminance relationship. Boundary tests should include negative values, zero, both sides of joins, super-white, extreme exposure, and non-finite inputs. Uniform random sampling of the normal range cannot guarantee coverage of these rare but important locations. Plots should use constant precision consistent with the paper's declared formulas.

A.3 An order for reviewing the source

Enter TranscodeJob through CLI or MainWindow, then follow ToolTrust, Probe, Cadence, FloatBridge, FloatTransformer/CudaTransformer, MetadataWriter, PixelSanity, and publication. This places each module in its actual lifecycle. Alphabetical browsing can miss cross-file conditions such as who holds the lease, who owns the partial, and when samples may be overwritten.

The accompanying repository manifest records the path, category, line count, and SHA-256 of every controlled file, with generated PTX and the third-party JSON single header distinguished. Manifest hashes are calculated from actual local file bytes. Git newline conversion may make them differ from repository-object content hashes, so they are not Git blob IDs. Readers should verify the commit first, then match the analysis materials against the manifest, avoiding a mixture of source and tests from different revisions.

A.4 Reading the experimental records

The regression XML is the result of the reported run. The sustained-test JSON contains conversion duration, the application-private-memory sequence, and sampling error. Historical performance JSON comes from repository version 1.2.0. Keeping these datasets separate preserves temporal and version provenance. Theoretical curves are generated directly from formulas by the typesetting script, not fitted to camera captures or presented as actual waveform-monitor readings.

The accompanying LaTeX and Markdown are editable sources, while the PDF is generated by a local vector-typesetting program. A platform-directory error in the built-in LaTeX compiler prevents validation of the complete project, so the source is not labeled as successfully built by that compiler. PDF formulas are vector paths, the body is searchable, and figures and tables carry numbers, units, data provenance, and scope limitations.

Appendix B. Repository Files and Corresponding Research Chapters

The following index covers every version-controlled file at the fixed revision. It identifies each file's role in implementation, testing, building, or documentary evidence, supplemented by a reproducible per-file line-count and hash manifest. Third-party libraries, generated code, images, and licenses are not treated as handwritten project algorithms. The index is a traceability table of material scope and implementation responsibility, not a statement that every program line has been formally proved.

Table 5. All 102 version-controlled files and their roles

File path and extent	Implementation responsibility / research use
.clang-format 9 lines	Repository formatting, line endings, and ignore rules; reproduction infrastructure
.gitattributes 10 lines	Repository formatting, line endings, and ignore rules; reproduction infrastructure
.github/workflows/build-windows.yml 71 lines	C++20/MSVC targets, test registration, build and release allowlists; Chapters 8, 21
.gitignore 48 lines	Repository formatting, line endings, and ignore rules; reproduction infrastructure
CHANGELOG.md 141 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
CMakeLists.txt 124 lines	C++20/MSVC targets, test registration, build and release allowlists; Chapters 8, 21
CONTRIBUTING.md 19 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
LICENSE 21 lines	Project or third-party licensing and attribution; distribution boundaries
README.md 233 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
THIRD_PARTY_NOTICES.md 41 lines	Project or third-party licensing and attribution; distribution boundaries
docs/APPLE_LOG_IDENTIFICATION.md 153 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/ARCHITECTURE.md 147 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/BENCHMARK_1.2.0.md 62 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/BUG_AUDIT_1.1.1.md 63 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/COLOR_PIPELINE.md 97 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/CREATIVE_ADJUSTMENTS.md 46 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic

File path and extent	Implementation responsibility / research use
docs/FFMPEG_PROVIDER.md 54 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/ICON.md 28 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/METADATA.md 79 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/RELEASE_1.2.0.md 92 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/RELEASE_1.2.1.md 46 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/TECHNICAL_REFERENCES.md 137 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/VALIDATION.md 598 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/benchmarks/1.2.0.json 2805 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/images/LogForge.png Binary: 37255 bytes	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
docs/verification/1.2.0.json 320 lines	Project documentation, historical evidence, or development procedures; cross-checked by chapter topic
include/logforge/AppleLogIdentification.h 14 lines	Writing the 35-byte identification extension and checking structure; Chapter 15
include/logforge/Color.h 53 lines	Color-model interfaces, constants, parameters, and statistics; Chapters 3-7
include/logforge/CudaTransformer.h 24 lines	Dynamic driver interface, qualification, and transactional commit; Chapter 11
include/logforge/Discovery.h 61 lines	Local volume and directory traversal; Chapter 17
include/logforge/FFmpeg.h 62 lines	Tool capabilities, discovery scheduling, and fixed downloads; Chapter 17
include/logforge/FloatBridge.h 21 lines	Three-slot read/transform/write state machine and thread budgeting; Chapter 9
include/logforge/FloatTransformer.h 18 lines	Persistent CPU workers, tile pairing, and statistics merging; Chapter 10
include/logforge/Localization.h 34 lines	Stable message keys and single-pass argument replacement; Chapter 20
include/logforge/Media.h 87 lines	Probe parsing, admission, metadata arguments, and output validation; Chapters 2, 16
include/logforge/Messages.inc 262 lines	Bilingual message macro table, stable keys, and placeholders; Chapter 20
include/logforge/PixelSanity.h 24 lines	Nine-block gross-error checks on first, middle, and last encoded frames; Chapter 7
include/logforge/Platform.h 115 lines	Wide strings, handles, process supervision, pipes, and hashing; Chapter 18
include/logforge/Queue.h 8 lines	Serial multi-file tasks and per-item failure recording; Chapter 20

File path and extent	Implementation responsibility / research use
include/logforge/Settings.h 23 lines	Field merging, concurrency locks, and atomic preference persistence; Chapters 19, 20
include/logforge/StorageSafety.h 18 lines	File-identity journals, state locks, and ownership-based recovery; Chapter 19
include/logforge/ToolTrust.h 29 lines	Identity, approval, and leases for the executable pair; Chapter 17
include/logforge/Transcode.h 33 lines	Complete media task, backends, validation reports, and final publication; Chapters 8, 23
include/logforge/Ui.h 49 lines	Themes, DPI, native drawing, and screenshots; Chapter 20
resources/LogForge.ico Binary: 103909 bytes	Icons, manifests, version resources, and generated templates; build and interface
resources/LogForge.manifest 9 lines	Icons, manifests, version resources, and generated templates; build and interface
resources/LogForge.png Binary: 1161472 bytes	Icons, manifests, version resources, and generated templates; build and interface
resources/LogForge.rc 34 lines	Icons, manifests, version resources, and generated templates; build and interface
resources/Version.h.in 6 lines	Icons, manifests, version resources, and generated templates; build and interface
resources/VersionResource.h.in 4 lines	Icons, manifests, version resources, and generated templates; build and interface
src/AppleLogIdentification.cpp 194 lines	Writing the 35-byte identification extension and checking structure; Chapter 15
src/Cadence.cpp 168 lines	All-packet period, phase, and fixed-clock validation; Chapter 12
src/Cli.cpp 223 lines	Commands, numeric arguments, and automation entry points; Chapter 20
src/Discovery.cpp 141 lines	Local volume and directory traversal; Chapter 17
src/DiscoveryHelper.cpp 501 lines	Supervised hidden discovery entry point and source budgets; Chapter 17
src/FFmpeg.cpp 416 lines	Tool capabilities, discovery scheduling, and fixed downloads; Chapter 17
src/FFmpegNumeric.cpp 234 lines	Independent integer signals and actual codec numerical qualification; Chapters 5, 7
src/FloatBridge.cpp 183 lines	Three-slot read/transform/write state machine and thread budgeting; Chapter 9
src/Localization.cpp 77 lines	Stable message keys and single-pass argument replacement; Chapter 20
src/MainWindow.cpp 1710 lines	Single background worker, events, and measured layout; Chapter 20
src/Media.cpp 400 lines	Probe parsing, admission, metadata arguments, and output validation; Chapters 2, 16
src/MediaSafety.cpp 272 lines	Time semantics, display matrices, and fixed-width header repair; Chapters 13, 16
src/MetadataPolicy.cpp 53 lines	Metadata allowlists and conflict classification; Chapter 16

File path and extent	Implementation responsibility / research use
src/MovAnalyzer.cpp 433 lines	Bounded atom parsing and typed semantic comparison; Chapter 14
src/PixelSanity.cpp 184 lines	Nine-block gross-error checks on first, middle, and last encoded frames; Chapter 7
src/Platform.cpp 398 lines	Wide strings, handles, process supervision, pipes, and hashing; Chapter 18
src/Queue.cpp 49 lines	Serial multi-file tasks and per-item failure recording; Chapter 20
src/Settings.cpp 120 lines	Field merging, concurrency locks, and atomic preference persistence; Chapters 19, 20
src/SettingsWindow.cpp 659 lines	Settings drafts, save/cancel, candidate review, and dialogs; Chapter 20
src/StorageSafety.cpp 214 lines	File-identity journals, state locks, and ownership-based recovery; Chapter 19
src/ToolTrust.cpp 124 lines	Identity, approval, and leases for the executable pair; Chapter 17
src/Transcode.cpp 584 lines	Complete media task, backends, validation reports, and final publication; Chapters 8, 23
src/Ui.cpp 322 lines	Themes, DPI, native drawing, and screenshots; Chapter 20
src/color/AppleLog.cpp 19 lines	Piecewise Apple Log encoding/decoding and public constants; Chapter 4
src/color/ColorKernel.cu 58 lines	Device-side double color kernel and shared-memory reduction; Chapter 11
src/color/ColorKernel.manifest.json 14 lines	CUDA source, PTX, compiler identity, and precise options; Chapter 11
src/color/ColorKernel.ptx.h 1620 lines	Generated PTX byte array: content-hash and source-manifest checks; not handwritten algorithm code; Chapter 11
src/color/CudaTransformer.cpp 333 lines	Dynamic driver interface, qualification, and transactional commit; Chapter 11
src/color/FloatTransformer.cpp 127 lines	Persistent CPU workers, tile pairing, and statistics merging; Chapter 10
src/color/HLG.cpp 98 lines	Inverse HLG, reference scaling, creative adjustment, and signal counts; Chapters 3, 6
tests/application.cpp 262 lines	Preferences, discovery, and version-resource tests; Chapter 21
tests/benchmark.py 97 lines	Formal throughput matrix and process observations; Chapter 22
tests/compatibility_v121.py 145 lines	Portrait pixels, matrices, and audio-offset regression; Chapters 13, 21
tests/cuda_artifact.py 18 lines	CUDA source, embedded PTX hashes, and option consistency; Chapter 11
tests/fake_ffmpeg.cpp 10 lines	Marker fixture establishing non-execution of unknown tools; Chapter 17
tests/gui_smoke.py 123 lines	Interactive desktop GUI scenarios; not rerun for this paper; Chapter 20
tests/hardening.cpp 367 lines	Cadence, tool trust, MOV, parallelism, and publication regressions; Chapter 21
tests/identification.cpp 101 lines	Identification writing, idempotence, and structural-failure tests; Chapters 15, 21

File path and extent	Implementation responsibility / research use
tests/integration.py 255 lines	End-to-end synthetic media; the reported run fails at the old admission assertion on line 154; Chapter 21
tests/layout_snapshots.py 35 lines	Production layout, theme, and DPI screenshot assertions; Chapter 20
tests/package_audit.py 38 lines	Archive allowlist, CRC, executable, and SHA-256 audit; Appendix
tests/process_fixture.cpp 51 lines	Descendant-held pipes, long lines, and exit-code fixtures; Chapter 18
tests/reliability.cpp 246 lines	Process, discovery, and CLI abnormal-path regressions; Chapter 21
tests/resolve_identification.py 110 lines	Real Resolve controlled-import script; not rerun for this paper; Chapter 15
tests/signal_integrity.py 160 lines	Independent integer patches, creative processing, and range counts; Chapters 7, 21
tests/sustained.py 90 lines	240-frame 4K120, memory, and post-encoding luma; Chapter 21
tests/tests.cpp 396 lines	Basic mathematical, media, and platform tests; Chapter 21
tests/v120.cpp 386 lines	Storage, matrices, CUDA faults, queues, and media regressions; Chapter 21
third_party/NVIDIA-CUDA-NOTICE.txt 23 lines	Project or third-party licensing and attribution; distribution boundaries
third_party/nlohmann/LICENSE.MIT 21 lines	Project or third-party licensing and attribution; distribution boundaries
third_party/nlohmann/json.hpp 25526 lines	Third-party JSON 3.12.0 single-header dependency for JSON I/O; not the project color algorithm
tools/compile_cuda.py 48 lines	NVRTC compilation with precise options and generated manifest; Chapter 11

References and Source Locations

Implementation discussion relies primarily on the fixed-revision source. Chapter-end source locations identify the principal files. The following list provides project material and key external primary references. External documentation is used to check public definitions and interfaces; historical repository experiments describe observations made at the time. Neither substitutes for the execution results reported in this paper. Web references were checked on September 29, 2026.

[1] Project source and README at the fixed revision.

<https://github.com/HongHuo0728/LogForge/tree/6b43a93fbd4d1457ff20b4c99a030c0e398f520a>

[2] LogForge COLOR_PIPELINE: numerical contract.

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/COLOR_PIPELINE.md

[3] LogForge CREATIVE_ADJUSTMENTS: project-defined adjustments.

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/CREATIVE_ADJUSTMENTS.md

[4] LogForge ARCHITECTURE: architecture and historical policies.

<https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/ARCHITECTURE.md>

[5] LogForge RELEASE_1.2.1: current admission and raster-orientation policy.

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/RELEASE_1.2.1.md

[6] LogForge VALIDATION: versioned historical validation records.

<https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/VALIDATION.md>

[7] LogForge BENCHMARK_1.2.0: historical performance matrix.

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/BENCHMARK_1.2.0.md

[8] LogForge APPLE_LOG_IDENTIFICATION: native references and controlled Resolve experiments.

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/APPLE_LOG_IDENTIFICATION.md

[9] LogForge METADATA and FFMPEG_PROVIDER: container and tool policies.

<https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/METADATA.md>

[10] LogForge TECHNICAL_REFERENCES: index of external primary references.

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/TECHNICAL_REFERENCES.md

References and Source Locations (continued)

- [11] Apple-supplied ACES IDT, fixed commit 528c78fe: Apple Log decoding constants.
https://github.com/ampas/aces-dev/blob/528c78fe2c0f4e7eb322581e98aba05de79466cb/transforms/ctl/idt/vendorSupplied/apple/IDT.Apple.AppleLog_BT2020.ctf
- [12] ITU-R BT.2100-3 (2025): HLG transfer function; official version index checked for this study.
<https://www.itu.int/rec/R-REC-BT.2100>
- [13] ITU-R BT.2408-6: HDR production reference; scaling is cross-explained against the fixed repository contract.
https://www.itu.int/dms_pub/itu-r/opb/rep/R-REP-BT.2408-6-2023-PDF-E.pdf
- [14] ITU-R BT.2020: UHD television primaries and matrix.
<https://www.itu.int/rec/R-REC-BT.2020>
- [15] FFmpeg official filter manual: zscale; concrete behavior is constrained by qualification tests.
<https://ffmpeg.org/ffmpeg-filters.html#zscale>
- [16] FFmpeg n8.1.2 MOV muxer source: the fixed version referenced by the repository.
<https://github.com/FFmpeg/FFmpeg/blob/n8.1.2/libavformat/movenc.c>
- [17] Apple QuickTime File Format: original matrix and container documentation.
<https://developer.apple.com/documentation/quicktime-file-format/matrices>
- [18] NVIDIA NVRTC 12.8.1: interface reference for PTX generation.
<https://docs.nvidia.com/cuda/archive/12.8.1/nvrtc/index.html>
- [19] Microsoft Windows: Job Objects supervision interface.
<https://learn.microsoft.com/en-us/windows/win32/procthread/job-objects>
- [20] CMake CPack: archive and checksum-generation reference.
<https://cmake.org/cmake/help/latest/module/CPack.html>